

Task Scheduling with Mobile Robots—A Systematic Literature Review

Catarina Rema ^{1,*}, Pedro Costa ^{1,2,†}, Manuel Silva ^{1,3,†} and Eduardo J. Solteiro Pires ^{1,4,†}

¹ INESC TEC—Institute for Systems and Computer Engineering, Technology and Science, 4200-465 Porto, Portugal; pedrogc@fe.up.pt (P.C.); mss@isep.ipp.pt (M.S.); epires@utad.pt (E.J.S.P.)

² Faculty of Engineering, University of Porto (FEUP), 4200-465 Porto, Portugal

³ ISEP, Polytechnic of Porto, rua Dr. António Bernardino de Almeida, 4249-015 Porto, Portugal

⁴ School of Science and Technology, University of Trás-os-Montes and Alto Douro (UTAD), 5000-811 Vila Real, Portugal

* Correspondence: catarina.rema@inesctec.pt

† These authors contributed equally to this work.

Abstract: The advent of Industry 4.0, driven by automation and real-time data analysis, offers significant opportunities to revolutionize manufacturing, with mobile robots playing a central role in boosting productivity. In smart job shops, scheduling tasks involves not only assigning work to machines but also managing robot allocation and travel times, thus extending traditional problems like the Job Shop Scheduling Problem (JSSP) and Traveling Salesman Problem (TSP). Common solution methods include heuristics, metaheuristics, and hybrid methods. However, due to the complexity of these problems, existing models often struggle to provide efficient optimal solutions. Machine learning, particularly reinforcement learning (RL), presents a promising approach by learning from environmental interactions, offering effective solutions for task scheduling. This systematic literature review analyzes 71 papers published between 2014 and 2024, critically evaluating the current state of the art of task scheduling with mobile robots. The review identifies the increasing use of machine learning techniques and hybrid approaches to address more complex scenarios, thanks to their adaptability. Despite these advancements, challenges remain, including the integration of path planning and obstacle avoidance in the task scheduling problem, which is crucial for making these solutions stable and reliable for real-world applications and scaling for larger fleets of robots.



Academic Editor: Sunan Huang

Received: 30 March 2025

Revised: 18 May 2025

Accepted: 26 May 2025

Published: 30 May 2025

Citation: Rema, C.; Costa, P.; Silva, M.; Pires, E.J.S. Task Scheduling with Mobile Robots—A Systematic Literature Review. *Robotics* **2025**, *14*, 75. <https://doi.org/10.3390/robotics14060075>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: task scheduling; task planning; mobile robots

1. Introduction

The rise of Industry 4.0 has unlocked significant opportunities to transform manufacturing processes through real-time data analysis, artificial intelligence (AI), automation, and the interconnection of production line components, all contributing to the creation of adaptable and resilient smart manufacturing environments. Among the key technologies driving this transformation, autonomous mobile robots (AMRs) have emerged as pivotal players. Their integration into, among others, job shop settings and hospitals for transportation and material handling tasks holds great potential for enhancing productivity and operational efficiency [1].

When considering AMRs, task scheduling has long been acknowledged as a complex optimization problem. The introduction of mobile robots adds new dimensions to this challenge, extending the traditional scope.

Task scheduling is formally defined as the problem of assigning a set of tasks to a set of robots, where the goal is to optimize various objectives, such as minimizing total completion time, balancing robot workloads, or reducing travel distances. Unlike task allocation, which focuses on deciding which robot should perform which task, task scheduling specifically addresses the timing and sequencing of tasks once the allocation is made. This includes determining the order in which tasks should be performed and managing the robots' movements between tasks to ensure optimal efficiency [2].

In a smart job shop, optimization now involves not only assigning tasks to machines but also allocating tasks to AMRs while accounting for their travel times. This added complexity positions the problem as a hybrid of the Traveling Salesman Problem (TSP) and the Job Shop Scheduling Problem (JSSP), which makes the scheduling process even more intricate [1].

The JSSP with AMRs involves two main NP-hard subproblems: mobile robot scheduling and machine scheduling. Various approaches, including heuristics, metaheuristics, hybrid methods, and optimization techniques, have been explored to address this problem. Heuristic algorithms typically focus on assigning transportation tasks to robots based on task pools, and metaheuristics aim to provide near-optimal solutions within acceptable time limits by exploring the solution space more thoroughly. Hybrid methods combine elements of both heuristics and metaheuristics to leverage the strengths of both approaches, improving solution quality and efficiency. Optimization techniques, such as integer programming and constraint programming, focus on finding the best possible solution by modeling the problem mathematically and searching for the optimal configuration. However, due to the problem's combinatorial complexity and the dynamic nature of manufacturing environments, existing models often face challenges in delivering optimal solutions efficiently [3]. Machine learning techniques are increasingly gaining attention, as they adapt and learn from system behavior, providing flexible, data-driven approaches capable of optimizing task scheduling in evolving and uncertain environments [4].

Given the growing interest in AMRs for job shops, hospitals, and other environments, task scheduling with mobile robots has become an area of active research. This paper presents a systematic literature review of existing approaches for task scheduling in environments involving AMRs, following the Preferred Reporting Items for Systematic Reviews and Meta-Analysis (PRISMA) methodology [5]. This systematic approach ensures the process is replicable for researchers using the same paper selection scheme, facilitating future updates and consistency in the field. The methodology led to the inclusion of 71 relevant works, which are discussed and analyzed in terms of algorithms, challenges, evaluation metrics, and emerging trends. By synthesizing these findings, this review aims to provide a comprehensive understanding of the current state of research and identify potential areas for future exploration.

The main contributions of this paper, compared to existing work in related areas such as task allocation for Multi-Robot Task Allocation (MRTA), are as follows:

- A thorough review of task scheduling, specifically in the context of mobile robots, addressing both task scheduling and task allocation, as well as task planning in multi-robot systems, which are often inter-related but underexplored in the existing literature;
- Critical gaps in the existing literature were identified, highlighting areas that are yet to be fully explored and offering directions for future research.

The remainder of this review is structured as follows: Section 2 provides an overview of existing works within the scope of this review and presents the main research questions addressed by this paper. Section 3 describes the methodology used to search for the relevant literature and select the records included in this review. In Section 4, the bibliographic

information of the 71 selected works is analyzed, identifying research networks, keywords, and trends related to the topic. This section also discusses the coverage of data sources considered in the methodology and presents the evolution of publications over time. Section 5 focuses on the methodologies of the 71 records related to task scheduling with mobile robots, including subtopics such as robot configuration, communication architecture, a categorization of implemented methods, and an additional analysis of works involving path planning and/or collision avoidance. Furthermore, this section includes an analysis of the experimental data, such as the size of robotic fleets used, tested scenarios, and the main evaluation metrics employed by the authors. Section 6 discusses the main limitations of this study, and finally, Section 7 answers the key research questions of this review while suggesting directions for future research in task scheduling with mobile robots.

Task Scheduling vs. Vehicle Routing Problem

Task scheduling for mobile robots and Vehicle Routing Problems (VRPs) are both combinatorial optimization problems that, while sharing some structural similarities, are fundamentally different in goals, constraints, and suitable solution approaches. Understanding these distinctions is key to modeling and solving the robotic task scheduling problem accurately.

A VRP involves planning routes for a fleet of vehicles that must deliver goods to a set of customers. Each vehicle starts and ends at a depot, and each customer must be visited exactly once. Vehicles may have capacity limits, and customers may have time windows during which deliveries must be made. The primary objective is to minimize the total travel distance (or, sometimes, the number of vehicles used). In classic VRP formulations, visit durations are negligible, and no task precedence constraints exist [6].

In contrast, the robotic task scheduling problem considered in this work involves a set of mobile robots that must execute a set of spatially distributed tasks. Each task (Table 1) has a specific location and a non-zero duration and may have precedence constraints (i.e., one task must be completed before another can begin). It requires exclusive access to an AMR and possibly shared resources (such as tools or workstations). Robots move between tasks, so travel time between locations is included in the model; however, the main objective is to minimize the total execution time (makespan), not travel distance. Additionally, robots can perform only one task at a time, and tasks cannot be interrupted once started [2,6].

Table 1. Comparison of vehicle routing and robotic task scheduling.

Feature	Vehicle Routing Problem	Robotic Task Scheduling
Objective	Minimize total travel distance	Minimize makespan (total completion time)
Task duration	Typically zero	Significant, non-zero
Task precedence constraints	Rare (except in variants)	Common and essential
Resource constraints	Vehicle capacity	Exclusive use of robots and shared tools
Spatial component	Core (customers and depot)	Core (task locations)
Resource mobility	Mobile vehicles	Mobile robots
Transition/travel time	Dominant factor	Considered but not primary
Time windows	Common	Common
Pre-emption	Not typical	Not allowed

2. Purpose of the Study

Recently, the topic of task scheduling for fleets of mobile robots has received significant attention, driven by the increasing deployment of mobile robots and the growing need to efficiently organize tasks for ever larger fleets. The main studies that review task scheduling and related themes, including mobile robots, are summarized in Table 2. In the first two studies [7,8], the JSSP is the main focus of the review. The topic of JSSP with mobile robots is only briefly mentioned in a subsection. Both studies highlight that this area is still underexplored by researchers. The review in [9] presents a gap in the task scheduling theme, as the search criteria used did not adequately focus on the subject. In contrast, the review in [10] concentrates exclusively on task allocation, neglecting other relevant areas such as task scheduling and task planning for multi-robot systems, which are often used together. The paper focuses solely on task scheduling and does not address the allocation of tasks to robots. While it is acknowledged that task allocation is an important consideration in multi-robot systems, the primary emphasis of this work is on the scheduling of tasks. This distinction is crucial because task scheduling involves determining the order and timing of tasks to optimize system performance, whereas task allocation is concerned with assigning specific tasks to individual robots.

Table 2. Existing literature reviews and surveys on task scheduling with mobile robots.

Reference	Survey Type	Year	Potential Gaps	Subject
[7]	General	2021	Yes	JSSP
[8]	General	2022	Yes	JSSP
[9]	Systematic	2023	Yes	Task allocation
[10]	Systematic	2024	Yes	Task allocation

Table 2 indicates that no systematic reviews have been conducted on task scheduling with fleets of mobile robots, particularly when considering quality assessment as a factor. The approach used in this systematic review is unique, offering a comprehensive and up-to-date overview of the past decade of research.

The repeatability of the systematic review methodology employed here allows future updates to the discussion and the methods presented. In addition, this literature review does not focus on any specific trends related to task scheduling with fleets of mobile robots, which enhances the breadth and coverage of the topic. In summary, this review aims to address the following research questions:

- What are the most commonly used technologies/algorithms for task scheduling with mobile robots?
- What are the main challenges and limitations of existing task scheduling techniques for mobile robots?
- Are artificial intelligence (AI)-based approaches emerging as a trend in this field?

3. Methodology

A systematic literature review employs explicit, rigorous, and reproducible methods to synthesize and analyze studies relevant to a specific research question, topic, or phenomenon of interest. This approach ensures that the review process is transparent and consistent, allowing for the reliable integration of findings from various sources. By organizing and summarizing the key findings from all relevant studies, this review not only provides a comprehensive analysis but also offers a robust framework that can be replicated or updated by future researchers. The most common standard for conducting a systematic review is the PRISMA statement [5]. Although it was initially designed for

evaluating the effects of health interventions, the checklist items of the methodology are general and applicable to other subject areas.

This section outlines the methodology employed in this systematic review. First, the eligibility criteria are defined to establish the studies included in the review. The search strategy follows, specifying the information sources considered, the base query string used, and the search fields applied in each source to perform the inquiry. The paper selection process, following the PRISMA statement, emphasizes quality evaluation to ensure that only relevant and high-quality studies are included in the synthesis and analysis phase. Finally, the data extraction process is detailed, describing the data collected for synthesis and analysis. The literature review methodology was supported by the Parsifal [11] online tool, which aids in designing the protocol, eliminating duplicates, and screening and selecting studies, including their quality assessment. The NotebookLM [12] tool was used for the data extraction phase.

3.1. Eligibility Criteria

The eligibility criteria for this systematic review, presented in Table 3, were defined to ensure the inclusion of studies that are highly relevant and offer solid evidence in relation to the research questions. The eligibility criteria were designed to focus on studies that are readily accessible in digital libraries, ensuring that all the papers included in this review are available for others to access. Limiting the review to studies published in English enhances its reach and visibility within the global research community. Regarding the year range, only studies published between 2014 and 2024 were included. This period was chosen because, as shown in Figure 1, since 2014, there has been a significant increase in the number of studies, reflecting exponential growth in the field, which justifies the focus on this period for the analysis.

Another exclusion criterion applied in this review pertains to the scope of the studies. Any studies that did not focus specifically on the theme of task scheduling were excluded. The final exclusion criterion relates to the type of robots studied, excluding papers that did not focus on Automated Guided Vehicles (AGVs) or autonomous mobile robots (AMRs). For example, studies involving Unmanned Aerial Vehicles (UAVs) were not considered in this review.

Table 3. Exclusion criteria for the selection process.

E#	Criteria	Statement
E1	Availability	Full text of the papers not available in digital libraries
E2	Language	Full text of the papers not published in English
E3	Year	Papers not included in the year range of [2014–2024]
E4	Scope	Papers that focus on different and not related subjects
E5	Robot Type	Papers that do not focus on mobile robots (AGVs or AMRs)

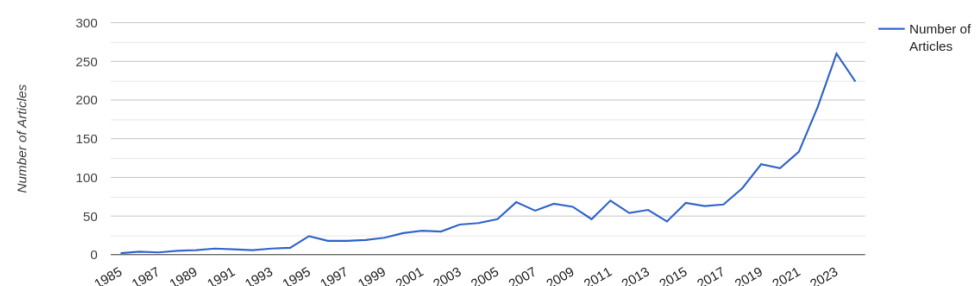


Figure 1. Number of existing articles per year on all searched digital libraries in this study.

3.2. Search Strategy

The search phase involves identifying relevant data sources for this literature review. Although Web of Science and Scopus are commonly used bibliographic databases, they do not encompass the full breadth of scientific coverage. Therefore, the data sources considered for this review include the ACM Digital Library, IEEE Xplore, Web of Science, and Scopus.

The date 24 October 2024 is the last full inquiry. Future reviews on the topic should have this data as a reference for the beginning of their work. The base string used for searching the data sources is (“mobile robots”) AND (“task planning” OR “scheduling”).

The first term of the query “mobile robots” refers to the population defined in the PICO template. Only the term “mobile robots” was used to narrow the search results, ensuring that they remain within the intended scope and focus on the specific type of robots relevant to this review.

The final part of the query focuses on the intervention aspect of the literature review. Given the goal of studying the task scheduling process, the terms (“task planning” OR “scheduling”) were selected. Many studies do not explicitly use the term “task scheduling” but instead refer to it as “task planning” or simply the scheduling of robots and machines. The term “planning” was avoided on its own to prevent confusion with “path planning”, which is a different concept. To ensure that relevant papers were included, “task planning” was preferred. Additionally, the use of the asterisk in “scheduling” allows for the inclusion of all synonyms related to the desired intervention, such as “scheduling” or “schedule”.

The title, abstract, and keywords were the fields selected to obtain the search results. The selection of these search fields for this review improves the relevance of the results compared to using all fields and searching in the full text since the main contributions of scientific works should be summarized in at least the title, abstract, or author keywords. The results relative to each search field were imported into Parsifal to further remove duplicates.

3.3. Selection Process

The selection process for this literature review is illustrated in Figure 2 and consists of three phases: identification, screening, and quality assessment. In the first phase, each data source mentioned earlier was queried using the search string. The second phase involved screening the papers by reviewing their titles and abstracts in Parsifal, followed by a decision on eligibility based on the established exclusion criteria. The third phase entailed evaluating the studies selected in the second phase by applying specific quality assessment criteria. The records that met the criteria across all three phases were then accepted for the data extraction phase.

3.3.1. Identification

To begin the identification phase, the search query was entered into four selected databases: ACM Digital Library, IEEE Xplore, Web of Science, and Scopus. This search resulted in a total of 3112 records, which were then exported to Parsifal for further analysis. The records were distributed as follows: a total of 747 from ACM Digital Library, 1000 from IEEE Xplore, 980 from Scopus, and 385 from Web of Science. Once imported into Parsifal, duplicates were removed, with 895 duplicates identified. Following this process, 2217 records remained for the next phase of screening.

3.3.2. Screening

The screening phase involved reviewing the title and abstract of each publication and excluding those that met the predefined exclusion criteria. Of the 2217 records, 2023 were excluded, leaving 194 publications to proceed to the quality assessment phase.

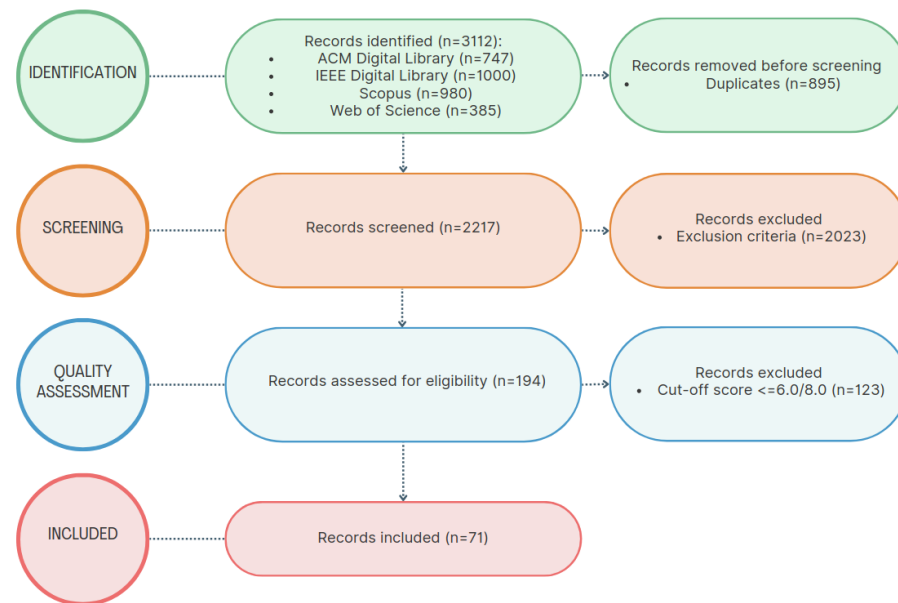


Figure 2. Flow diagram for the selection process.

3.3.3. Quality Assessment

After the screening, the next step was the quality assessment phase, where the publications were evaluated based on a set of predefined criteria. The specific evaluation criteria used in this review are outlined in Table 4. These criteria are framed as questions that the paper must address. Each criterion is assigned a score: 0 for a “No” response, 0.5 for a “Partially” response, and 1.0 for a “Yes” response. The total score for each publication can range from 0 to a maximum of 8 points.

QE1, QE2, and QE4 focus on the level of detail provided in the papers. Specifically, they assess whether the discussion of related work, the proposed methodology, the experimental setup, and the results are comprehensive and thoroughly analyzed, respectively. QE3 aims to determine whether the study involves a single robot or multiple robots and whether there is any coordination between the agents. QE5 evaluates whether the work has been compared with other ground truth studies to validate the proposed methodology. QE6 builds upon QE4 by further validating whether the study is sufficiently detailed to allow for potential replication. QE7 examines whether the results are evaluated through simulation or real-world experiments to confirm the accuracy and validity of the proposed approach. Finally, QE8 assesses the scalability of the method, specifically its ability to be implemented with more robots and tasks.

Table 4. Quality evaluation criteria and score range.

QE#	Criteria	Score
QE1	Does the paper have an updated state of the art on task scheduling techniques?	{0.0, 0.5, 1.0}
QE2	Is the methodology appropriate and clear?	{0.0, 0.5, 1.0}
QE3	Does the study address multi-robot coordination or cooperation?	{0.0, 0.5, 1.0}
QE4	Is the hardware and/or software well-detailed in the methodology?	{0.0, 0.5, 1.0}
QE5	Does the paper compare the implemented methodology with any other methods?	{0.0, 0.5, 1.0}
QE6	Can the study be replicated based on the provided information?	{0.0, 0.5, 1.0}
QE7	Is the performance of these algorithms evaluated in a meaningful way (e.g., through simulations or real-world experiments)?	{0.0, 0.5, 1.0}
QE8	Is the solution scalable?	{0.0, 0.5, 1.0}

The quality assessment phase involves setting a cut-off score to filter out papers with low quality. To visualize the score distribution, the diagram in Figure 3 was created. Papers with scores below 6.0 begin to exhibit deficiencies, such as a lack of information (e.g., no mention of hardware or software used), single-robot implementations, non-scalable solutions, or an absence of comparisons with other methods. These papers are considered unsuitable for inclusion in this study. Therefore, a score of 6.0/8.0 or higher was used as the threshold to exclude such works. After the quality assessment evaluation, 71 papers were selected to be included in this work for the next phase, data extraction.

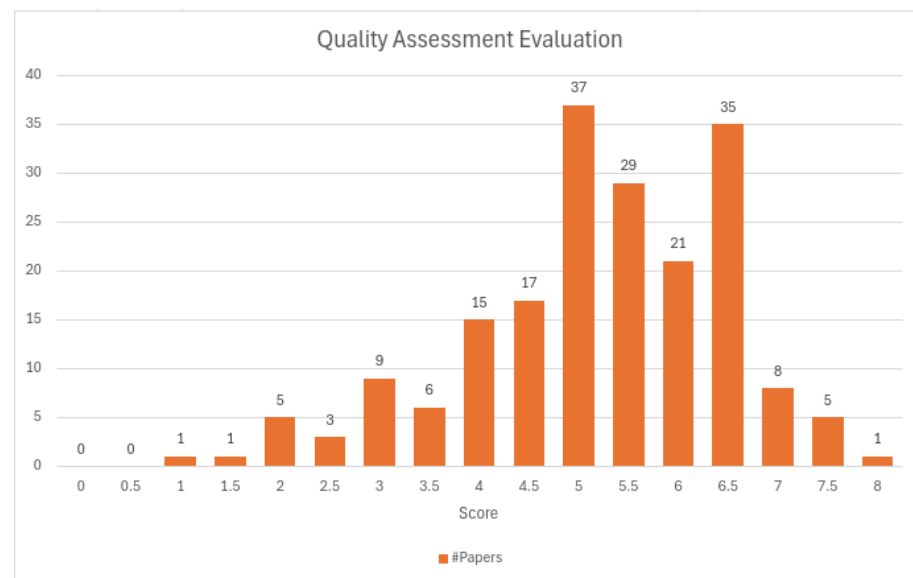


Figure 3. Quality assessment evaluation diagram.

3.4. Data Extraction

The data extraction process analyzes the records selected after the quality assessment phase and extracts relevant information from these studies. For the purpose of this review, the required data extraction (DE#) items are as follows:

- DE1 Robot Configuration—Whether the study focuses on a single-robot or multi-robot setup.
- DE2 Communication between Robots—The type of communication used, such as centralized, distributed, or no communication mentioned.
- DE3 Algorithm—The name of the algorithm or approach utilized in the study.
- DE4 Algorithm Category: The classification of the algorithm, such as heuristic, metaheuristic, hybrid, or other types.
- DE5 Algorithm Adaptations—The main modifications and contributions made to the algorithm, differentiating it from previous versions.
- DE6 Evaluation Metrics—The performance metrics used by the authors to evaluate the proposed approach.
- DE7 Test Environment—Whether the experiments were conducted in simulations or real-world settings, including details on the hardware and software used, as well as the specific experiments performed.
- DE8 Comparison with Baselines—The algorithms or approaches used for comparison with the proposed method.
- DE9 Study Limitations—Any potential limitations identified.

Data extraction from the included records was performed using the NotebookLM software. The PDF files were uploaded into the software, and specific data extraction items were defined. The software then provided responses to each item based on the content

of the papers. Although the tool is highly useful, it requires post-processing verification, and all outputs were manually checked and confirmed through direct examination of the papers.

4. Results Overview

In this section, the bibliographic information of the 71 selected works is analyzed, identifying research networks, keywords, and trends related to the topic. This section also discusses the coverage of data sources considered in the methodology and presents the evolution of publications over time.

4.1. Data Source

In the search strategy phase, four data sources were utilized, and the search string for this review was defined. During the identification phase, the results were exported from these data sources in BibTeX format and were subsequently imported into the Parsifal tool. This export included all available information from each data source, such as citation details (author, title, and publication venue), bibliographic information for each record, abstract, author, and indexed keywords. Afterward, a Python script was employed to check and identify incomplete records, such as missing DOIs.

Considering the 71 records included in this review following the selection process, the following details the numerical and percentage distribution of records corresponding to each data source:

- ACM Digital Library [13]: 2 records (2.82%);
- Scopus [14]: 41 records (57.75%);
- IEEE Digital Library [15]: 18 records (25.35%);
- Web of Science [16]: 10 records (14.08%).

Scopus is the database with the highest number of records included in this review, accounting for over 50%, due to its status as one of the largest databases. IEEE follows in second place, contributing 25.35%. As a digital library, IEEE indexes only works published by IEEE and its partners. Web of Science represents 14.08% of the included records. Although Web of Science is a highly impactful database like Scopus, many studies from this source were found to be duplicates of records from other databases, which explains its relatively low contribution in this review. Finally, the ACM Digital Library includes just two papers, representing 2.82% of the records. Similar to IEEE, ACM only indexes works published by its own organization, with a primary focus on computing rather than engineering and robotics, which accounts for its lower representation compared to the other databases.

4.2. Keywords Co-Occurrence

VOSviewer [17,18] was utilized to analyze keyword co-occurrence within the selected articles, exploring the relationships between terms based on their simultaneous occurrence across documents. Initially, a Python script was used to process the BibTeX file containing citation and bibliographic details, merging author names and keywords into a single unified keyword field. The processed BibTeX file was then converted into RIS format using an online tool.

While VOSviewer supports direct input from sources like Dimensions, Scopus, and Web of Science, it does not natively support BibTeX, which necessitated the conversion to RIS format.

The co-occurrence analysis was performed using two different frequency thresholds: a minimum of four occurrences (Figure 4) and a more stringent threshold of five (Figure 5). These configurations allowed us to investigate the impact of varying frequency thresholds

on the structure of the network and provided valuable insights into the main themes of the included studies. The keywords identified in the networks closely reflect and relate to the query outlined in Section 3.2. The most prominent keywords are *mobile robots* and *scheduling*, which are the core components of the query. Terms such as *genetic algorithms*, *heuristic algorithms*, *reinforcement learning*, and *integer programming* show the relevant methods used, with the term *makespan* as the main goal and *manufacture*, *warehouses*, *transportation* as the main applications.

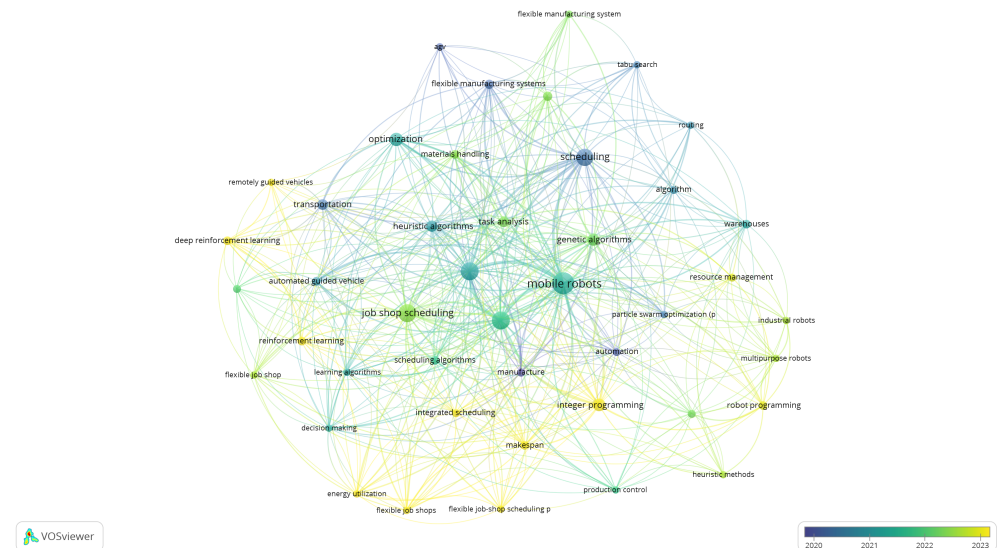


Figure 4. Keywords co-occurrence analysis on the 71 included records generated by VOSviewer with overlay visualization according to the average publication year. The parameters used for generating the co-occurrence network are as follows: minimum number of occurrences = 4, attraction = 2, repulsion = 0, scale = 1.51, circle size variation = 1.00, and line size validation = 1.00.

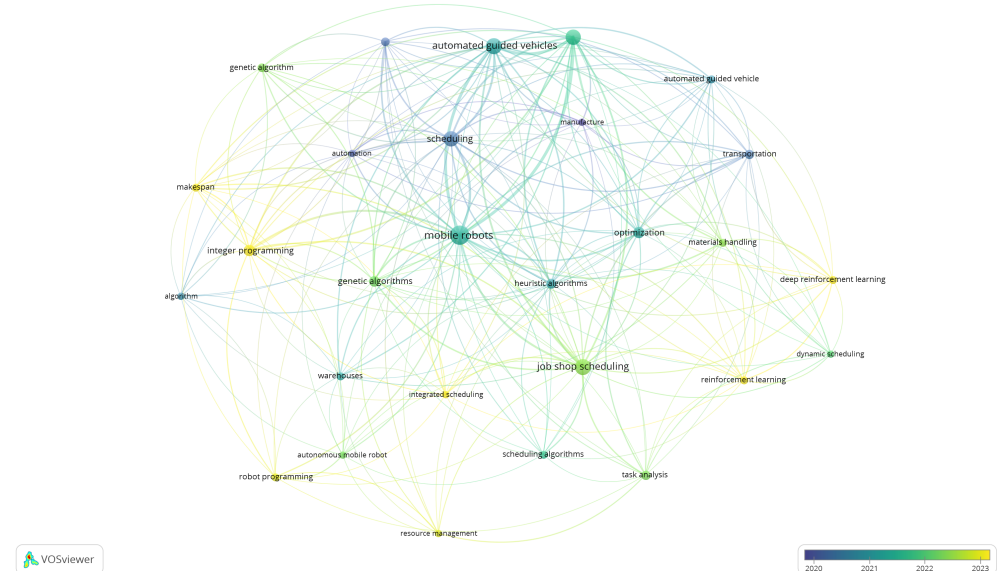


Figure 5. Keywords co-occurrence analysis on the 71 included records generated by VOSviewer with overlay visualization according to the average publication year. The parameters used for generating the co-occurrence network are as follows: minimum number of occurrences = 5, attraction = 2, repulsion = 0, scale = 1.51, circle size variation = 1.00, and line size validation = 1.00.

The overlay visualization displays the co-occurrence network of keywords weighted by their frequency. The size of each circle represents the number of occurrences of a keyword,

and the strength of the links between terms reflects their co-occurrence frequency. The color gradient indicates the average publication year, with lighter colors representing more recent terms. Using a threshold of three occurrences, this network provides a comprehensive overview, capturing diverse terms and their relationships.

When comparing the two previous figures, Figure 4, which has four as the minimum number of occurrences, captures a broader landscape of terms, whereas Figure 5, with five minimum number of occurrences, focuses on more dominant terms in the included works.

A more detailed analysis of Figure 5 reveals that terms such as *reinforcement learning* and *deep reinforcement learning* have appeared more prominently in recent years, highlighting the increasing prominence of these new technologies and their growing applications in the field of task scheduling.

4.3. Year of Publication

The distribution of articles included in this systematic review shows considerable variation over the years, as presented in Figure 6. No articles were included from 2014. The number of articles gradually increased, with three in 2015, one in 2016, two in 2017, and two in 2018. Between 2019 and 2021, the number of included articles remained consistent and also increased, with seven records in 2019, five records in 2020, and eight in 2021. Only three records were included in 2022. The years 2023 and 2024 saw the highest number of included papers, with 20 articles each. This trend reflects the growing body of research on the topic of task scheduling with mobile robots, highlighting the importance and relevance of this review to the scientific community.

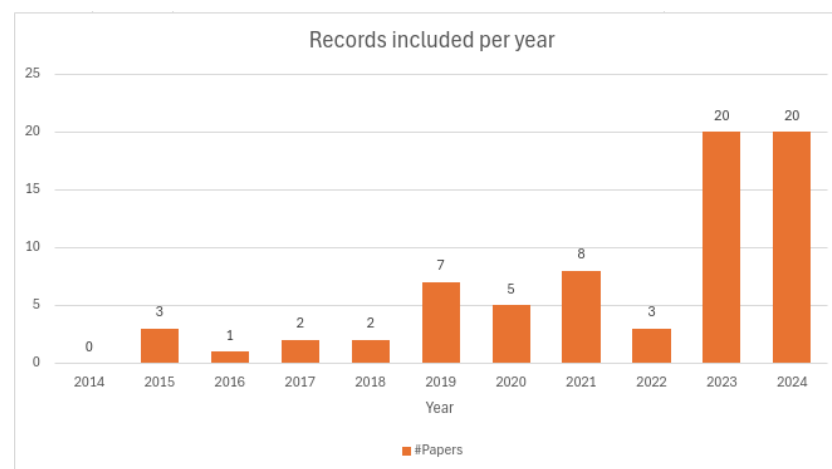


Figure 6. Records included per year.

5. Discussion

The primary objective of this review is to gather a synthesized analysis focused on task scheduling with mobile robots in one document. Therefore, this section provides a comprehensive overview of the methodologies and algorithms employed in the existing literature, categorizing them into distinct sections for better clarity and analysis.

First, Section 5.1 outlines the various robot configurations and communication protocols. Next, the review explores Sections 5.2–5.6, where different task scheduling techniques are examined in detail, discussing the specific algorithms and methodologies used. Section 5.7 then addresses the integration of path planning algorithms and obstacle avoidance strategies, which are crucial for ensuring optimal task execution in dynamic environments.

Finally, Section 5.8 provides a thorough analysis of the experiments conducted in each of the reviewed works, evaluating the performance of the proposed methodologies and

discussing the results. Additionally, it is important to note that in the tables presented in this chapter, the symbol “X” indicates that a particular aspect was considered in the corresponding study. This structure ensures a holistic understanding of the task scheduling problem with mobile robots, offering insights into current advancements and identifying gaps for future research.

5.1. Robot Configuration and Communication Methods

When performing task scheduling with mobile robots, an important factor to consider is the type and number of robots used. In this review, only multi-robot systems are included, with a score cut-off of 6.0/8.0, obtained according to Section 3.3.3 to exclude single-robot configurations. For fleets of mobile robots, the robots can either be homogeneous or heterogeneous. Homogeneous robots have identical hardware, software, and capabilities, while heterogeneous robots vary in hardware (e.g., dimensions), software, capabilities, or a combination of these factors.

Task scheduling for homogeneous robots is generally straightforward, as all robots share the same tasks and capabilities. In contrast, scheduling for heterogeneous robots can involve multiple approaches, such as organizing tasks based on the robots’ capabilities or balancing their workloads.

Coordination between robots becomes critical when dealing with fleets larger than two agents. This review investigates two coordination techniques: centralized and distributed. In centralized coordination, all agents are “connected” to a central system, which can be advantageous for smaller systems but may not be practical in large-scale applications. For larger systems, a distributed approach, where coordination occurs directly between agents without a central system, is more appropriate [19].

A detailed analysis of these features is presented in Table 5 for the included records.

Table 5. Quality evaluation of agent types and coordination methods.

Reference	Agents		Coordination	
	Homogeneous	Heterogeneous	Centralized	Distributed
[19]	X			X
[20]	X		X	
[21]	X		X	
[22]	X		X	
[23]	X		X	
[24]	X		X	
[25]	X		X	
[26]	X		X	
[27]	X		X	
[28]	X		X	
[29]	X		X	
[30]	X		X	
[2]	X		X	
[31]	X		X	
[32]	X			X
[33]		X	X	
[34]	X		X	
[35]	X		X	
[36]		X	X	
[37]	X		X	
[38]		X	X	
[3]	X		X	

Table 5. Cont.

Reference	Agents		Coordination	
	Homogeneous	Heterogeneous	Centralized	Distributed
[39]	X		X	
[40]	X		X	
[41]	X		X	
[42]		X	X	
[43]	X		X	
[44]	X		X	
[45]	X		X	
[46]	X		X	
[47]	X		X	X
[48]	X		X	X
[49]	X		X	
[50]		X	X	
[51]	X		X	
[52]	X		X	
[53]	X		X	
[54]	X		X	
[55]	X		X	
[56]	X			X
[57]	X		X	
[58]	X		X	
[59]	X		X	
[60]	X			X
[61]	X		X	
[62]	X		X	
[63]	X			X
[64]	X		X	
[65]	X		X	
[66]	X		X	
[67]	X		X	
[68]	X		X	
[69]	X		X	
[70]	X		X	
[71]	X		X	
[72]	X		X	
[73]	X		X	
[74]	X		X	
[75]	X		X	
[76]	X		X	
[77]	X		X	
[78]		X	X	
[79]	X		X	
[80]	X		X	
[81]	X		X	
[82]		X	X	X
[83]	X		X	
[84]	X		X	X
[85]		X	X	
[86]	X		X	
[87]	X		X	

5.2. Heuristic Methods

Heuristic algorithms are problem-solving techniques that yield approximate solutions with computational efficiency, sacrificing optimality for speed. In task scheduling for

mobile robots, they help efficiently allocate tasks by considering factors like time and distance, making them useful for complex or real-time problems. Looking at Table 6, only 8 out of 71 records in this review are based on heuristic methods, showing their relatively limited presence in the literature. The table lists all the included records that employ heuristic methods, along with their respective objective functions and the algorithms used.

Table 6. Objective functions and associated algorithms/methods in heuristic approaches from included records.

Reference	Objective Function	Algorithm/Method
[26]	Minimize makespan	Greedy approach
[19]	Minimize makespan	Sequential Distributed RHTA (S-DRHTA) and Negotiation-based Distributed RHTA (N-DRHTA)
[41]	Minimize average cycle time	Dynamic programming-based heuristic algorithm
[56]	Minimize makespan	Allen's interval algebra
[60]	Minimize makespan	Regularized quadratic programming approach
[67]	Minimize total cost	Heuristic scheduling model
[83]	Minimize makespan	Two-Stage Heuristic Algorithm (TSHA)
[87]	Minimize waiting time	Clarke and Wright Savings Algorithm (CWSA)

A range of heuristic approaches have been applied to task scheduling. Avhad et al. [26] used a Greedy approach to allocate transport robots (TRs) based on the shortest distance during the initialization phase of production. TRs were assigned tasks based on cost, with the lowest-cost TR receiving the task. Although rack sorting was not explicitly mentioned, tasks were re-bid in FIFO order to ensure task allocation even when TRs were unavailable. This heuristic solution proved effective for managing dynamic tasks in a production setting, although it might lack the optimality found in more formal methods. Chen and Liu [19] employed two distributed Robot Task Allocation (RHTA) methods, Sequential Distributed RHTA (S-DRHTA) and Negotiation-based Distributed RHTA (N-DRHTA), which operate in a decentralized manner. These methods assign priority to tasks or form coalitions for negotiation, depending on the approach. Tasks were grouped into “petals” and assigned based on battery power and time constraints, highlighting a real-time, adaptive scheduling system in dynamic environments. Kim et al. [41] used a Markov Decision Process (MDP) model for a single Automated Mobile Robot (AMR) and extended this to a heuristic algorithm for managing multiple AMRs. The algorithm considered battery charging constraints when selecting workstations and operations, a critical factor in mobile robot systems where energy efficiency plays a key role. Mudrova and Hawes [56] formalized the task scheduling problem as a Mixed-Integer Programming (MIP) problem and used Allen's interval algebra to reduce the MIP search space. This approach, though not purely heuristic in nature, is relevant because it combines heuristic pruning techniques with an optimization framework, enabling faster computation by narrowing down the search space. Their method helps eliminate impossible execution orders by analyzing time windows and selecting optimal task pairs based on time relations. Bakshi et al. [60] took a different approach by relaxing constraints through doubly stochastic matrices to approximate permutation matrices in task scheduling. They introduced a regularization term to guide solutions toward valid permutations using a gradient-based optimization method. Though primarily an optimization method, the heuristic nature of their approach lies in the iterative projection and computational efficiency it offers for real-time applications. Yi and Luo [67] developed two heuristic functions, hMPD (maximum potential difference) and hTPD (total potential difference), for process sorting and AGV routing. By

using task-APF (Artificial Potential Field) and place-timed Petri Nets (PNs), these functions predict task completion times and guide task assignment. The combination of task-APF and route-APF in their approach makes it particularly well-suited for dynamic, constraint-driven environments. Yu et al. [83] focused on the TSHA algorithm to solve the multi-load AGV scheduling problem in automated sorting warehouses. Their two-phase heuristic first grouped similar packages based on a clustering algorithm and then used a load-balancing scheduling algorithm to assign these groups to AGVs. This dual-phase heuristic approach aims to optimize AGV usage and reduce travel distances, providing an efficient solution to real-time logistical challenges. Dang et al. [87] introduced two batching methods for picklists: Regular Heuristic (RH) and Savings Heuristic (SH). The RH method reflected the current method for generating pickruns, while SH optimized batch creation by minimizing travel distances. The SH method identified savings by merging picklists and reducing total travel distances, improving efficiency in warehouse order picking.

While heuristic algorithms provide flexible, quick solutions to task scheduling problems, their effectiveness varies depending on the specific system constraints and the environment in which they are applied. These algorithms are typically most effective in small-to medium-scale systems, where adaptability and real-time decision-making are prioritized over finding globally optimal solutions.

5.3. Metaheuristic Methods

Metaheuristic methods are advanced search techniques that guide the process of finding near-optimal solutions to complex problems. In mobile robot task scheduling, they explore large solution spaces more effectively than simple heuristics, often using strategies inspired by nature or optimization processes to improve results over time. In Table 7, 24 out of 71 records in this review are based on metaheuristic methods, demonstrating their significant presence in the literature. The table lists all the included records that employ metaheuristic methods, along with their respective objective functions and the algorithms used.

Table 7. Objective functions and associated algorithms/methods in metaheuristic approaches from included records.

Reference	Objective Function	Algorithm/Method
[21]	Minimize the makespan	Multi-strategy-driven Genetic Algorithm (Mult stra GA)
[22]	Minimize total cost	Multi-Restart Iterated Greedy (MRIG) algorithm
[25]	Minimize the makespan	Adaptive Large Neighborhood Search (ALNS) algorithm guided by item characteristics (I-ALNS)
[28]	Minimize the makespan and energy consumption	Efficient Heuristic Algorithm (EHA)
[29]	Minimize the makespan	Collaborative Dual-Population Genetic Algorithm (DPGA)
[2]	Minimize makespan and energy consumption	Improved Multi-Objective Differential Evolution (IMODE) algorithm
[40]	Minimize makespan	Symbiotic Organisms Search Algorithm (SOSA)
[42]	Minimize makespan	Discrete Particle Swarm Optimization (DPSO)
[45]	Minimize makespan	Bio-inspired Scheduling Optimization Approach (BSOA)
[49]	Minimize makespan and work imbalance	Fuzzy Multi-Objective Invasive Weed Optimization (FMOIWO) + Fuzzy Multi-Objective Cuckoo Search (FMOC) algorithms
[50]	Minimize makespan and battery consumption	Differential Evolution-Fused Particle Swarm Optimization (DEFPSO)
[51]	Minimize energy consumption	Modified Genetic Algorithm (MGA)
[52]	Minimize makespan	Ant Colony Optimization (ACO)

Table 7. *Cont.*

Reference	Objective Function	Algorithm/Method
[53]	Minimize makespan	Genetic Algorithm (GA) + Particle Swarm Optimization (PSO)
[57]	Minimize makespan	Adaptive Large Neighborhood Search (ALNS)
[62]	Minimize makespan	Knowledge-Based Multi-Objective Evolutionary Algorithm (KBMOEA)
[65]	Minimize makespan	Particle Swarm Optimization (PSO) algorithm
[70]	Increase tasks allocated and minimize battery consumption	Island Model Genetic Algorithm (IMGA)
[73]	Minimize makespan	Chaotic sparrow search algorithm
[77]	Minimize makespan	Integrated Local Search Probability-based Memetic Water Cycle (LSPM-WC) algorithm
[78]	Minimize makespan	Adaptive Large Neighborhood Search (ALNS)
[79]	Minimize total cost and travel time	Discrete Jaya (DJaya) algorithm
[80]	Minimize makespan	Learning-based multipopulation evolutionary optimization (LMEO)
[81]	Minimize makespan	Genetic Algorithm (GA)

Genetic algorithms (GAs), originally introduced by John Holland [88], mimic the process of natural selection. They use crossover, mutation, and selection operators to evolve a population of candidate solutions. GAs are popular for their robustness in exploring large search spaces and for their ability to avoid local optima when properly tuned. Li et al. [21] proposed a Multisearch GA to solve a task scheduling problem with mobile robots. The strategy combined several advanced techniques to enhance optimization performance. It used a two-layer-based encoding and decoding approach to represent solutions more effectively. A multiple heuristics-based initialization ensured diverse and high-quality initial solutions. The double crossover operator promoted genetic diversity by combining parent solutions in multiple ways, while dual mutation operators introduced randomness to prevent premature convergence. Han et al. [29] introduced a new Mixed-Integer Linear Programming (MILP) model formulated to obtain optimal solutions for small-scale instances. Additionally, DCGA was designed to efficiently solve the FJSP-AGV problem. The DCGA was based on GAs and incorporated a two-layer encoding strategy and two different decoding methods used in two subpopulations to expand the solution space. Furthermore, a population collaboration operation was designed to enhance the search capability of DCGA. Wang et al. [51] incorporated a model that factored in the number of depots and the battery consumption of AGVs. It introduced a map complexity coefficient to account for the distance between workstations, considering the area, distribution, and shape of obstacles in the environment. To solve this, an MGA was used, with specific adjustments to ensure that AGV battery consumption aligned with task requirements. For single-depot scenarios, the MGA ensured battery usage was managed, while for multiple depots, a post-processing step directs AGVs to charging areas when battery levels were low after task completion. The map complexity coefficient enhanced the scheduling by evaluating the obstacles' total area, distribution, and shape characteristics. Pan et al. [80] used an LMEO algorithm for task scheduling. This algorithm enhanced solution search by incorporating several innovative strategies. It divided the population into elite, exploratory, and exploitative subpopulations, improving diversity and convergence. The algorithm combines heuristics and random methods for high-quality initialization and uses Q-learning to select parents for the evolutionary search. It incorporates problem-specific local search operators for refining machine and AGV assignments and scheduling. A statistical learning approach was applied for population replacement, maintaining elite individuals while improving diversity. Additionally, a three-string encoding scheme represented scheduling,

machine assignment, and AGV assignment. The GA employed by Wang and Xin [81] used a chromosome composed of three parts: operation sequence, machine selection, and AGV selection. A decoding process converted this chromosome into a schedule. Genetic operators like precedence operation crossover (POX) for operations and partially matched crossover (PMX) for machines and AGVs were applied. Mutation for operations used an insertion operator, while machine and AGV mutation randomly replaced a gene with a different machine or AGV. A tournament selection operator was used to choose individuals for reproduction.

Iterated Greedy (IG) algorithms iteratively destruct and reconstruct solutions to explore neighborhoods around good candidates. Liu et al. [22] used the MRIG algorithm, which is based on the IG algorithm. MRIG enhanced this approach with several advanced strategies to improve solution optimization and avoid getting trapped in local optima. These improvements included an enhanced initialization strategy (INNH) that went beyond a basic greedy approach, a multi-restart strategy with various methods to diversify the search and escape local optima, and a destruction phase with an adaptive parameter. Additionally, MRIG featured two improved reference local searches (RLSs) to intensify the search for better solutions and an acceptance criterion that incorporates simulated annealing concepts, allowing occasional acceptance of worse solutions to escape local minimal.

ALNS, first introduced by Ropke and Pisinger [89], iteratively destroys and repairs parts of a solution, adapting operators over time to balance intensification and diversification. Qin et al. [25] proposed the ALNS algorithm guided by item characteristics (I-ALNS). In the first stage, an item similarity and popularity-based method (ISPDm) was used to obtain the sequencing of orders and totes across workstations, while in the second stage, a GA was employed for robot scheduling, directing robots to handle all totes according to the predefined sequence. Dang et al. [57] used ALNS to improve the solution by employing a combination of destroy and repair heuristics. The destroy heuristics removed parts of the current solution to create space for potential improvements, including random removal (removing elements randomly), worst removal (removing elements that contribute the most to the makespan, such as tasks on the critical path), and related removal (removing tasks that are closely related, like those assigned to the same robot or occurring near each other in time). The repair heuristics then rebuilt the destroyed parts of the solution, aiming to enhance its overall quality. This iterative process, where the destroy and repair steps were adapted to the problem context, enabled ALNS to effectively solve complex scheduling problems. Song et al. [78] developed an approach based on ALNS, using destruction and repair operators to explore the solution space and avoid local optima. Destruction operators removed tasks from the current solution, including random, worst, worst route, and Shaw destruction. Repair operators then reinserted tasks using strategies like greedy insertion and regret-k, which prioritized cost-effective reintegration.

Xu et al. [28] proposed the EHA, which is based on NSGA-II, addressing the multi-objective green scheduling problem of flexible job shops with AGVs. It used a two-layer solution encoding, where the operation sequence and machine assignments were represented. A high-quality initial population was generated considering processing times and energy consumption, while a greedy insertion decoding method selected AGVs and maximized machine usage. The algorithm employed two crossover methods (POX for the operation sequence and MPX for machine assignment) and two mutation methods (SIM for sequence inversion and MRM for machine replacement). Additionally, a local search strategy based on critical paths optimized the solutions by minimizing makespan on the Pareto front. The work of He et al. [2] presented the IMODE algorithm designed to solve the Energy-efficient Open-shop Scheduling Problem with multiple AGVs and deteriorating jobs (EOSSPA). IMODE incorporated several advanced components, including a four-layer

encoding scheme to represent operation sequences, machine speed, AGV sequences, and AGV speed. It featured a population initialization mechanism based on a problem-specific heuristic and mean entropy theory to enhance initial population quality and diversity. The algorithm also employed a fitness evaluation mechanism using grey entropy parallel analysis with a dynamic reference point, differential evolution operators, and a multi-level local search strategy to improve local search performance. Mareddy et al. [40] adapted the SOSA to solve the problem of simultaneous scheduling of machines, AGVs, and tools with minimal tool copies and no tool delays. The solution was represented as a vector encoding job operations, AGV assignments, machine allocations, and tool selections. SOSA phases (mutualism, commensalism, and parasitism) explored the solution space, while random solution generation and limit functions ensured feasibility by respecting precedence constraints and operational limits. Gu et al. [45] proposed a methodology that used a biological analogy between a manufacturing system and the human endocrine system, where machines and AGVs acted like endocrine glands, and information exchange was akin to hormonal secretion. In this approach, machines secreted a “hormone” when a transport task arose, indicating urgency. AGVs calculated an evaluation score based on the hormone and their efficiency in completing the task, selecting the task with the shortest weighted completion time. Machines, in turn, assigned the task to the AGV with the lowest HS. The process was iterative, continuously allocating tasks until all transport duties were fulfilled. The methodology also adapted to disruptions, such as machine failures or urgent orders, by adjusting the schedule dynamically and ensuring flexibility. Nabovati et al. [49] combined hybrid optimization techniques, specifically the FMOIWO and FMOC algorithms. The method incorporated a novel chromosome structure to represent both machine and AGV schedules, ensuring feasible solutions. It accounted for machine breakdowns using predictions from an Artificial Neural Network (ANN), enhancing the robustness of the schedules. The multi-objective fitness function balanced multiple criteria such as makespan and AGV workload. Additionally, fuzzy logic was integrated to manage uncertainties like variations in processing and travel times. Saidi-Mehrabad et al. [52] used an ACO to optimize AGV scheduling by preventing conflicts, ensuring no AGV shared the same node, edge, or machine at the same time. It operated in two stages: Stage 1 focuses on job sequencing, where pheromone values were updated to favor sequences with better completion times. Stage 2 reinforced optimal routes, guiding AGVs towards machines and minimizing completion times by accumulating pheromones on more efficient paths. Yao et al. [62] introduced a new active decoding method to improve search efficiency by expanding the search space of the algorithm. This method allowed for the separate insertion of transport and processing tasks, which enhanced the algorithm’s ability to explore more diverse solutions. Additionally, four problem properties were passively incorporated into the local search strategy, leveraging domain knowledge to guide and improve the search process. An adaptive mechanism was implemented to balance exploration and exploitation by dynamically adjusting selection and mutation probabilities. Cechinel and Pierre [70] proposed a task scheduler based on the IMGA for coordinating heterogeneous mobile robots with varying payloads, energy consumption, and speeds. The main contribution of the scheduler was its ability to optimize task allocation by simultaneously considering multiple constraints, including task deadlines, robot payload capacity, and battery levels. This simultaneous consideration of constraints ensured that the scheduler efficiently managed the total load transported while minimizing battery consumption. Additionally, the scheduler guaranteed that robots meet their deadlines and battery requirements and could detect and notify of failures during task execution, supporting its online operational capability. Sun et al. [73] addressed the issue of local optimal solutions in the later stages of the Social Spider Algorithm (SSA) applied to FJSP-AGV. The authors proposed a three-layer coding approach with a greedy

decoding scheme using Hénon chaotic mapping for population initialization to avoid premature convergence. Krishnamoorthy et al. [77] suggested LSPM-WC combined with the Water Cycle Algorithm (WCA) and Local Search Probability-based Multi-Agent Algorithm (LSPMA). The LSPMA incorporated a local search probability (LSP) to select individuals for the next population, focusing on reducing production time in task scheduling. The WCA, inspired by the water cycle, was adapted to address the multi-load AGV scheduling problem, treating tasks as “raindrops” and searching for the “best location” or optimal solution. By integrating both algorithms, LSPM-WC leveraged the global exploration capabilities of WCA and the enhanced local search capabilities of LSPMA. Cui et al. [79] proposed adaptations to the Jaya algorithm for solving the scheduling of AGVs in delivery and unloading tasks. To handle the discrete nature of the problem, the Jaya algorithm was discretized, and two operators were introduced: the near-optimal operator, which moved solutions toward better ones by reinserting elements from the current solution into feasible positions to minimize cost, and the away worst operator, which moved solutions away from the worst ones to avoid errors.

PSO, introduced by Kennedy and Eberhart [90], is inspired by the social behavior of flocks and swarms. It works by updating the velocity and position of particles based on individual and group experience. Xiao et al. [42] improved classic PSO for AGV task scheduling by using a hybrid encoding scheme that combines task priority and execution mode. It employed dynamic matrices to represent task precedence and resource constraints. A decoding method was applied to transform particle codes into valid solutions, considering constraints like task order and resource availability. The fitness function aimed to minimize the total completion time of transportation tasks. Khosiawan et al. [50] combined Differential Evolution (DE) and PSO in a hybrid algorithm called DEFPSO. It started with an initial population generated using heuristic-based priority rules, seeding the search with promising solutions. In the PSO component, the position update was guided by a randomly selected particle from the current swarm to enhance exploration, while the velocity update incorporated a parameter for balancing exploration and exploitation. The DE component applied a crossover operation with the global best particle, refining solutions and improving convergence. Mousavi et al. [53] combined GA and PSO. GA was employed to explore a broad search space and avoid becoming trapped in local optima, while PSO was utilized to refine the solutions by efficiently converging towards the optimal solution. In this hybrid algorithm, GA first generated an initial population of solutions, and then PSO refined these solutions, enabling a more efficient and comprehensive search process to find the best possible solution. Li et al. [65] presented a dynamic adaptation mechanism and a bi-level coding method for handling order and task allocation. This method adjusted the algorithm to manage challenges such as out-of-bounds values, duplicated values, and ensuring feasible solutions. It effectively transferred PSO from a continuous to a discrete approach, making it suitable for combinatorial optimization problems. Additionally, the method introduced a local optimal strategy, where particles that fail to improve their performance are replaced and restructured with other particles. This adjustment helps avoid stagnation and ensures continued exploration of the solution space.

The reviewed studies collectively highlight the effectiveness of metaheuristic approaches in addressing the complex, multi-constraint scheduling problem involving machines and AGVs. Each method presents specific strengths: GAs offer versatility and integration with heuristics, IG excels at local refinement, and ALNS balances intensification and diversification. However, challenges such as computational cost, parameter sensitivity, and scalability persist across most methods. Future developments should emphasize adaptive, learning-guided, and real-time scheduling frameworks capable of han-

dling dynamic industrial conditions and heterogeneous task requirements with minimal human intervention.

5.4. Hybrid Methods

Hybrid methods refer to approaches that combine different techniques, algorithms, or methodologies to solve complex problems more effectively than any single method could on its own. In Table 8, 24 out of 71 records in this review are based on hybrid methods, highlighting their significant role in addressing task scheduling challenges for mobile robots. The table lists all the included records that employ hybrid methods, along with their respective objective functions and the algorithms used.

Table 8. Objective functions and associated algorithms/methods in hybrid approaches from included records.

Reference	Objective Function	Algorithm/Method
[20]	Minimize the adjusted average order cycle time (ACT)	Deterministic scheduling/routing algorithm + simulation optimization technique
[23]	Minimize the makespan	Variable Neighborhood Search (VNS) + ALNS
[27]	Minimize the makespan	GA and Tabu Search (TS) algorithm (GA-TS)
[30]	Analyze the peak capacity of a dual-robot in-line stocker (DRIS)	Analytical model based on a combined simulation and scheduling
[31]	Minimize the makespan and energy consumption	Hybrid genetic algorithm (HGA)
[32]	Minimize completion time	Game-theoretic decision-making algorithm integrated with a queuing system
[33]	Minimize makespan	Cooperative hybrid evolutionary algorithm (ChEA)
[34]	Minimize makespan	Self-learning genetic algorithm (SLGA)
[36]	Minimize makespan	Hybrid discrete state transition algorithm (HDSTA)
[37]	Minimize makespan	Multi-AGV scheduling approach considering assignment rules (MASA)
[44]	Minimize makespan	GA embedded with a rule-based solution construction method
[46]	Minimize makespan	Order Batch to Robot Task Conversion (OBRTC) algorithm + the Adaptive Robot Task Selection (ARTS) method
[47]	Minimize makespan	Benders decomposition
[48]	Minimize makespan	Integer Linear Programming (ILP)
[54]	Minimize makespan and makespan of the last job	Timed colored Petri nets (TCPN)
[55]	Minimize makespan	Shortest-Job-First (SJF) + TS
[59]	Minimize the makespan	GA-TS
[64]	Minimize makespan	GA with adaptive Fuzzy logic
[66]	Minimize makespan	GA and Reinforcement Learning (RL)
[69]	Minimize makespan	Two-step multiple-scenario approach (MSA)
[75]	Minimize makespan	Deep Q-Network (DQN) + Greedy algorithm
[76]	Minimize makespan	Modified Nomadic-Based Lion Algorithm (MN-LA)
[82]	Minimize makespan	MILP + GA
[84]	Minimize makespan and travel distance	Parallel Variable Neighborhood Search (PVNS)

Genetic algorithms (GAs) are known for their ability to perform global searches, and many studies have integrated them with other algorithms to enhance their performance and address scheduling and routing problems in AGVs and machines. Samsuria et al. [27] and Dang et al. [59] integrated the strengths of both a GA and TS, where the GA was responsible for generating the initial solutions or population through its operators, such as selection, crossover, and mutation. Concurrently, the TS focused on conducting neighborhood

search processes to refine and further explore promising areas within the search space. TS helped to intensify the search in the vicinity of high-quality solutions produced by the GA, thus improving the overall optimization process in each generation of the GA. Pradhan et al. [32] proposed an integrated strategy-based approach that combined a game-theoretic framework with a decentralized queuing system for mobile multi-robot task coordination. The decision-making algorithm utilized intricate reward and penalty values to guide the robots' actions. For motion coordination, the paper mentioned the use of the Potential Field Method along with Genetic Algorithm-Fuzzy algorithms. To select the best strategy among different coordination methods, game theory was applied, specifically utilizing the Maximin and Minimax methods to determine optimal strategies. Dou et al. [66] worked with GA for task scheduling and RL for path planning. The GA had a selection strategy that combined an elitist approach with the roulette wheel method. The top 20% of individuals were directly copied to the next generation to ensure convergence, while the remaining individuals were selected based on their fitness, with selection probability proportional to their fitness. For crossover, individuals were paired, and a variation of the order crossover operator was applied to combine their traits. For mutation, the algorithm used swap mutation, where an individual position was altered by swapping two elements, introducing genetic diversity to the population. Lu et al. [37] introduced two workstation assignment rules to optimize AGV operations: the crowding-based assignment rule (CAR), which assigned AGVs to workstations with the fewest waiting AGVs, and the opening-time-based assignment rule (OTAR), based on the workstation's expected opening time.

Variable Neighborhood Search (VNS) and Adaptive Large Neighborhood Search (ALNS) are powerful techniques used in hybrid algorithms to explore solution spaces more efficiently and effectively. In the work of Pang and Zhen [23], the authors introduced a hybrid heuristic approach that combined the VNS and ALNS algorithms, enhancing the solution space and improving solution speed. It employed an MILP model to minimize makespan. The ALNS algorithm adapted the weights of destruction and repair operators during iterations, allowing for more efficient exploration of the solution space. Meanwhile, VNS optimized this solution space specifically for the ALNS structure. Additionally, the group-swap method was utilized for neighborhood search strategies, further improving the algorithm's performance in finding optimal solutions. Zhang et al. [20] proposed a hybrid method integrating deterministic scheduling and routing algorithms with a simulation budget allocation procedure. The scheduling algorithm assigned orders to the nearest AGV within a specified range, while the routing algorithm ensured collision-free paths using constrained breadth-first search (BFS). An improved simulation budget allocation procedure reduced computational complexity and optimized algorithm parameters in a parallel computing environment. The integration of these methods enhanced overall system performance by optimizing both scheduling and routing decisions. For task scheduling in the cloud, Lin et al. [84] modeled the problem as a Colored Traveling Salesman Problem (CTSP), which was solved using a Parallel Variable Neighborhood Search (PVNS) algorithm. PVNS decomposed the initial CTSP solution into multiple TSP subproblems, which were processed in parallel by different threads using candidate set greedy insertion to obtain the final solution. This approach was implemented as a cloud service using Function-as-a-Service (FaaS) with OpenFaaS and Docker containerization, and Kubernetes managed the computing resources.

MILP, combined with heuristics, can handle the complexity of AGV and machine scheduling problems, offering powerful optimization capabilities in dynamic environments. Hu et al. [36] used an MILP model to analyze the scheduling of multiple AGVs, integrating both task assignment and path planning. To solve this complex problem, the authors introduced a hybrid discrete state transition algorithm (HDSTA) that incorporated an elite solution

set and TS to optimize the overall system. The HDSTA algorithm ensured high-quality solutions by maintaining a set of elite solutions, improving convergence, and helping escape local optima. The algorithm was tailored to account for different AGV capabilities and battery levels, and it used a decomposed approach inspired by previous work, separating the problem into an upper-level master problem and a lower-level subproblem, though the focus in this review was on the upper level. Liu et al. [34] presented the novel algorithmic framework, SLGA-D, which integrated a self-learning genetic algorithm (SLGA) with Dijkstra's algorithm with time window (DijkstraTW) to minimize travel time and makespan in a flexible and dynamic environment. The CFRP model was employed to minimize travel time, whereas the ISMV model focused on minimizing makespan. Additionally, two vehicle dispatching strategies, DSDS and FDS, were explored in the ISMV context to optimize vehicle routing and task scheduling. Zhang et al. [31] included a HGA that integrated an LNS to enhance its performance. The algorithm utilized single-point crossover and single-point mutation to generate offspring and then applied LNS to explore the neighbors of the selected individuals, further improving solution quality by refining the search in the solution space. Boysen et al. [69] proposed a two-step MSA for real-time problem-solving with limited lookahead, adapted to handle uncertainty in the arrival sequence of products. The approach separated the decision process into two steps: piece-to-order (P2O) assignment and order-to-collection point (O2CP) assignment, proving more effective than a single-step variant. Additionally, a heuristic decomposition approach (DEC) was introduced for the deterministic case, breaking the problem into three sequential decisions: P2O, O2CP, and piece-to-robot (P2R) assignment. For the P2O and O2CP subproblems, both MIP models and priority-rule-based heuristics were proposed, with First-Come-First-Served (FCFS) used for P2R. The HORS-MIP model simultaneously considered all three interdependent decisions, providing a holistic solution to the robotized sorting problem. Feo-Flushing et al. [82] combined an MILP model with a GA. The GA utilized problem-specific operators like best-subsequence mutation and mixed-subsequences crossover. Additionally, an anytime exact algorithm was developed, providing optimality bounds and leveraging a shared incumbent environment (SIE), where matheuristic and MILP solvers exchanged solutions. Riazi and Lennartson [47] presented an improved method for solving the conflict-free scheduling and routing of AGVs using Benders decomposition. It introduced several variations, including Benders-CP (using constraint programming), heur-Benders-CP (a heuristic version), Benders-Z3 (using disjunctive programming and the Z3 solver), and heur-Benders-Z3 (with a satisfiability subproblem). The method incorporated enhancements such as handling heterogeneous home locations, reformulating the subproblem for larger graphs, and introducing a heuristic decomposition to minimize makespan. It also considered heterogeneous travel times, improving both solution quality and speed. Mayer et al. [48] utilized a decentralized control system for AGV scheduling and routing. The Job routing agent used Monte-Carlo Tree Search (MCTS) for dynamic route selection based on workstation utilization, while the vehicle agent employed ILP for transport assignment, incorporating AGV coordination rules. This methodology adapted ideas from previous works.

RL, rooted in the trial-and-error learning paradigm, is particularly valuable in scheduling problems because it allows systems to adapt to dynamic environments in real time, optimizing AGV operations through continuous interaction and feedback. Xiaoting et al. [75] used a DQN agent to determine which operation to process at each decision point, using its learning capabilities to optimize scheduling decisions based on the current environment state. Once the operation was selected, a greedy algorithm was used to identify the most suitable machine for processing the operation and the appropriate AGV for transporting the job, ensuring efficient resource allocation. Additionally, the methodology included an experience replay mechanism, allowing the agent to learn from past experiences.

In addition to algorithmic approaches, simulation and analytical modeling techniques have been used to estimate system performance, guide decision-making, and complement scheduling algorithms. These methods often simulate material handling tasks and evaluate scheduling strategies under varying conditions. Chung [30] developed an analytical model that combined simulation and scheduling methods to estimate the transportation capacity of the DRIS under various conditions. The approach consisted of two main steps: first, simulation was used to randomly generate the origins and destinations of material handling tasks and to divide crossover tasks (those crossing the robot's operational zones) into two separate tasks. Second, scheduling calculated the times and locations of the two robots for each task, considering precedence constraints and using FIFO logic to select the next task. Bolu and Korcak [46] introduced a task conversion algorithm and heuristic method for task allocation to robots, aiming to improve efficiency by combining multiple tasks into a single operation. Unlike traditional methods that assign tasks one by one, this approach allowed robots to handle several orders at once. In the ARTS system, WES selected a task for an idle robot from the task pool based on three key parameters: (1) the distance of the order task's pod to the robot, (2) the time elapsed since the creation of the order task, and (3) the completion rate of the tote(s) assigned to the order task. Baruwa and Piera [54] presented a modeling approach using TCPN for the simultaneous scheduling of machines and AGVs, addressing two models: Machine Control (MCSS) and Vehicle Control (VCSS). The TCPN model used places and transitions to represent system states and events, with tokens indicating attributes like job IDs, machines, and AGVs. The hybrid heuristic search algorithm was employed to explore the state space efficiently, combining techniques like Breadth-First Iterative Deepening A* (BFIDA*), Suboptimal Breadth-First Branch-and-Bound (sBFBnB), and backtracking. Vivaldini et al. [55] developed a system that assigned tasks to AGVs using two heuristics: Shortest-Job-First (SJF) and Tabu Search (TS). SJF prioritized shorter tasks to minimize the average execution time, incorporating aging indices to prevent neglect of longer tasks. The two aging indices used were the ratio of the number of pallets in the dock to the maximum number of pallets and the ratio of total time spent by each AGV to the highest total time among AGVs. The TS metaheuristic explored the solution space by iterating over different mutations to improve task assignments and storing changes in a list to avoid revisiting previous solutions. In the work of Samsuria et al. [64], the innovation lies in the selection of fuzzy rules and measuring parameters based on the specified fitness function and its performance. Unlike other existing works, the input parameters in this method differed by incorporating both minimum and average fitness values as measures of GA performance. These fitness values were used to control the output parameters, specifically the crossover and mutation rates, enabling more adaptive and efficient search behavior in the algorithm. Abidi et al. [76] employed an intelligent model for predicting optimal scheduling rules in FMS, involving three main phases: feature extraction, optimal weighted feature extraction, and prediction. The model improved the conventional LA to optimize feature extraction with reduced correlation and enhanced a hybrid classifier (combining fuzzy logic and Deep Belief Networks (DBN)) by fine-tuning its membership functions, activation function, and the number of hidden neurons.

Other variants of evolutionary and swarm-based algorithms, including PSO and specialized crossover operators, have been applied to flexible job shop scheduling. Luo et al. [33] applied the ChEA algorithm to flexible job shop scheduling problems, combining a Gaussian PSO algorithm with a local search strategy based on the critical path method. The goal was to minimize manufacturing time by effectively optimizing task scheduling and resource allocation in the job shop environment. The authors Qu et al. [44] developed an approach that distinguished itself by considering parts and tools transfer time, unlike other works. It introduced the POX operator to facilitate the crossover process between two chromosomes. For Material Robots (MRs) assignment, a nearest vehicle

rule was applied, where the closest available MR was chosen to transport parts and tools. Additionally, a rule-based MR assignment algorithm was implemented to enhance the genetic process, ensuring more efficient allocation of MRs by avoiding a completely random assignment, leading to improved solution quality.

Hybrid algorithms that integrate techniques such as GA, MILP, RL, and VNS provide powerful solutions for AGV and machine scheduling by optimizing task allocation, routing, and resource use in dynamic environments. Despite their effectiveness, these approaches face challenges related to computational complexity, scalability, and, particularly, real-time applications. Future research should prioritize adaptability, efficiency, and self-learning capabilities to enhance responsiveness and performance in large-scale, real-world systems.

5.5. Machine Learning Methods

Machine learning (ML) methods have gained considerable attention in recent research on mobile robot task scheduling due to their ability to leverage historical data for decision-making, adapt to dynamic environments, and improve real-time task scheduling. These data-driven approaches are particularly useful in addressing complex, large-scale scheduling problems, where traditional algorithms may struggle to adapt quickly to changes in the environment. In Table 9, 8 out of 71 records in this review are based on machine learning methods, reflecting their emerging role in addressing task scheduling challenges. The table lists all the included records that employ machine learning methods, along with their respective objective functions and the algorithms used.

Table 9. Objective functions and associated algorithms/methods in machine learning approaches from included records.

Reference	Objective Function	Algorithm/Method
[35]	Minimize makespan	Deep Reinforcement Learning (DRL)
[39]	Minimize makespan and energy consumption	Deep Reinforcement Learning (DRL)
[43]	Minimize completion time	Reinforcement Learning (RL)
[61]	Minimize completion cost of orders	Deep Reinforcement Learning (DRL)
[72]	Minimize completion time and cost	Reinforcement Learning (RL)
[74]	Minimize makespan	Multi-Agent Proximal Policy Optimization with Global Critic (MAPPO-GC)
[85]	Minimize makespan	Deep Reinforcement Learning (DRL)
[86]	Minimize makespan	Deep Reinforcement Learning (DRL)

A clear trend in ML-based scheduling methods is the adoption of RL techniques. For example, Durst et al. [35] formulated their task scheduling problem as a Markov Decision Process (MDP), where the state, action representation, reward, and optimal policy function were defined in detail. They used Proximal Policy Optimization (PPO) combined with a Deep Neural Network (DNN) to optimize multiple objectives, including makespan, average machine utilization, and average job waiting time. The PPO algorithm was implemented using the TensorForce package based on TensorFlow, and a reward function was designed to optimize these objectives, with each factor assigned a corresponding discount factor. This approach showcases how reinforcement learning can be applied to multi-objective task scheduling, balancing various performance metrics in dynamic environments. Similarly, Zhang et al. [39] also tackled the AGV scheduling and battery replacement management problem by modeling it as an MDP. They employed a Dueling Deep Double Q Network (D3QN) to solve the problem. The system used historical experience to learn optimal policies, guided by a composite reward function that aimed to reduce production costs while meeting service demands. This hybrid approach integrated reinforcement learning with tra-

ditional optimization concepts, enhancing the scheduling of AGVs while addressing battery management efficiently. This highlights the flexibility of ML in combining traditional and modern optimization methods for complex scheduling tasks. Agrawal et al. [43] modeled the task scheduling problem from previous works as an MDP, where homogeneous agents shared a common policy. Their reward function incentivized efficient task scheduling by assigning positive rewards for completed machine jobs. Agents were further rewarded for completing subsequent machine tasks. This method focused on minimizing delays and optimizing throughput by encouraging agents to take the shortest paths for task execution. This approach demonstrates how ML can be applied to improve task efficiency and throughput in scheduling problems. Cheng et al. [61] proposed an algorithm that combined DRL with neural networks to optimize decision-making. Starting with random action selection, they incorporated a heuristic method to handle large action spaces and reduce training time. The state space included order allocations, which diminished over time to improve efficiency. The reward function was designed to minimize total system cost. This hybrid approach facilitated a more intuitive learning process and allowed the algorithm to iteratively improve by maximizing accumulated rewards, showcasing how combining heuristics with ML can accelerate learning in scheduling tasks. Zhao et al. [72] proposed a multi-agent attention model based on an encoder-decoder structure and an attention mechanism applied within a reinforcement learning framework for task scheduling in an AGV system. In this architecture, the encoder calculated initial node embeddings using a trainable weight matrix; these were then updated through multiple attention layers that consisted of multi-headed attention and a feed-forward network with skip connections and batch normalization. The final output from the encoder consisted of updated node and graph embeddings. The decoder took the encoder's output, along with a task completion mask and a context connection vector containing historical task information. It employed multi-head attention to compute task selection probabilities, and tasks were selected based on the highest probability. This method emphasizes the potential of attention mechanisms in multi-agent ML systems, allowing for more sophisticated task allocation strategies by capturing the temporal and spatial dependencies in the scheduling problem. Geng et al. [74] converted the scheduling task into an MDP. It consisted of three main agents: the Machine Selecting Agent (MSA), the Workpieces Sorting Agent (WSA), and the AGV Selecting Agent (ASA), each responsible for specific scheduling tasks. During training, the agents interacted with a simulated environment, observing states, taking actions, and receiving rewards based on their collective decisions, which were stored in a memory pool for experience replay. A global critic network evaluated the actions of all agents, promoting cooperative behavior and improving overall decision-making efficiency. The training process optimized key parameters, including learning rates, memory size, and batch size, using methods like Optuna optimization and manual fine-tuning to enhance performance. This multi-agent approach highlights the collaborative potential of machine learning, where different agents can specialize in various tasks while optimizing performance across the system as a whole. In the work of Ho et al. [85], a task scheduling approach for autonomous warehouses with heterogeneous robots was modeled as a queueing control optimization problem. The goal was to minimize task queue lengths, accounting for stochastic task flow and robot diversity. The Proximal Policy Optimization (PPO) method was used to find the optimal scheduling policy due to its strong performance in both discrete and continuous control. Furthermore, a decentralized PPO algorithm was proposed through Proximal Weighted Federated Learning (PWFL), which enhanced agent performance by considering system heterogeneity through a weighted aggregation of local models, where the weight was based on each agent's average reward. Finally, Li et al. [86] employed a DQN-based dispatching system to determine the optimal action for a dispatching agent in real time. By collecting

data on agent locations, routes, and task locations, the system used the DQN model to select the best movement decision. The Rainbow DQN framework, including Double DQN, Dueling DQN, and Distributional DQN, enhanced the model's decision-making capabilities. This approach showcases how deep Q-learning techniques can be applied to optimize task dispatching in real-time systems.

Machine learning methods, especially reinforcement learning and multi-agent systems, are increasingly being applied to mobile robot task scheduling problems. These methods offer significant advantages, such as adaptability to dynamic environments and the ability to optimize complex, multi-objective scheduling tasks. However, they also present challenges in terms of computational complexity and training time, which require ongoing research and refinement.

5.6. Optimization Methods

Optimization methods in mobile robot task scheduling are designed to find the best solution by minimizing critical factors such as time, energy, or cost. These methods rely on mathematical models and algorithms, such as MILP, Linear Programming (LP), or cooperative game theory, to systematically determine the most efficient way to allocate tasks. Unlike heuristic or metaheuristic approaches, which often provide approximate solutions, optimization methods involve structured problem-solving and formal mathematical frameworks, making them distinct in their approach. As shown in Table 10, 7 out of 71 records in this review specifically utilize optimization methods, illustrating their focused application in task scheduling. This table provides an overview of the included records, detailing their objective functions and the algorithms employed. Given the specialized nature of optimization methods, this section is separated to provide a focused discussion on these formal techniques, highlighting their unique role in task scheduling.

Table 10. Objective functions and associated algorithms/methods in optimization approaches from included records.

Reference	Objective Function	Algorithm/Method
[24]	Minimize the number of rack visits	Dynamic Programming algorithm
[38]	Minimize makespan and energy consumption	Capability-based robust task assignment and scheduling (CTAS) framework
[3]	Minimize makespan	MILP model based on the modified disjunctive graph model
[58]	Minimize makespan	Fuzzy-based scheduler
[63]	Minimize makespan	Maintenance-aware Task and Charging (MTC) algorithm
[68]	Minimize total cost	Contract-based Cyclic Motion Planning (CCMP)
[71]	Minimize makespan	Bargaining-game-based approach

Justkowiak et al. [24] proposed a dynamic programming algorithm where the state pruning technique combined dynamic programming with mathematical optimization, calculating a lower bound for the number of rack visits needed to reach the final state through a set cover problem solved by an MILP solver. States were pruned if the sum of current visits and the lower bound exceeded the best-known upper bound. Rack sorting was carried out based on priorities: first by the number of picks for incomplete orders, second by picks for unprocessed orders, and third by random order in case of ties. Subsets of orders that could be partially completed were sorted by cardinality, and duplicate orders were processed in a strict, arbitrary order to avoid redundancy. The algorithm used a depth-first search strategy to quickly improve the upper bound with pruning techniques. Fu et al. [38] modeled agent capabilities under uncertainty using vectors of random dis-

tributions. They introduced Conditional Value at Risk (CVaR) into the objective function to ensure robust task allocation in unpredictable environments. Their two-phase solution involved a risk-aware routing and scheduling model, followed by a flow decomposition subproblem, enhancing computational scalability. This work stands out for addressing uncertainty explicitly in the optimization process—an important consideration in real-world robotics. Yao et al. [3] presented an MILP model based on the modified disjunctive graph model. This formulation aimed to solve scheduling problems by incorporating both integer and continuous variables. The modified disjunctive graph model represented tasks and their dependencies, where the disjunctions (or alternatives) represented different possible sequences of tasks with certain constraints applied. In this context, the MILP model was used to optimize scheduling, where there were three subproblems: processing task sequencing, transport task assignment, and transport task sequencing. The disjunctive graph, in this case, helped to model the precedence relations between the tasks. Specifically, the transport task had to precede the processing task. Al-Momani and Al-Aubidy [58] employed simultaneous scheduling of machines and robots, considering both machine operations and robot battery levels. The MSA prioritized machines based on processing time, traveling distance, and the input buffer queue using fuzzy logic, with 27 rules to make decisions. The RSA incorporated the robot's battery charge and traveling distance, using fuzzy sets to ensure robots were assigned tasks while optimizing battery usage. The MTC scheduler, proposed by Atik et al. [63], used Linear Programming and the Kuhn–Munkres algorithm to allocate tasks to AMRs while considering energy levels and maintenance needs. It initialized a weight matrix based on factors like energy levels, task requirements, and battery degradation costs. The Kuhn–Munkres algorithm was employed to optimize task assignments, minimizing total costs. Additionally, decisions regarding when to start or stop charging were made based on AMR energy levels, ensuring efficient task allocation and energy management. This integration of energy-aware decision-making into a classic optimization algorithm highlights the growing attention to sustainability and maintenance in task scheduling. Leet et al. [68] proposed the CCMP methodology, which innovatively decomposed the Warehouse Scheduling Problem (WSP) into three phases: Traffic System Design, Traffic Cycle Set Synthesis, and Task Scheduling Synthesis. In the first phase, warehouse operators designed a traffic system with junctions connected by roads, setting high-level movement constraints for robots. The second phase utilized Assume–Guarantee contracts to calculate a traffic plan that specified the number of robots and products entering and leaving each junction and road during each cycle period, which was then decomposed into traffic cycles. In the final phase, task scheduling was performed by determining the allocation of robots to tasks, ensuring that product pickup and drop-off operations were completed efficiently across the traffic system. Their use of Assume–Guarantee contracts in traffic planning is a novel contribution, showcasing how optimization can be integrated into multi-layered system design for warehouse robotics. Wang et al. [71] presented an adaptation of a bargaining-game-based approach to solve a multi-objective optimization problem, distinguishing it from traditional methods like complete reactive scheduling, which relied on rules such as Shortest Processing Time and First-In-First-Out. In this approach, the bargaining game functioned as a cooperative game, where decision-makers negotiated the distribution of “profits” or, in this case, the optimal assignment of tasks to machines. The key advantage of using a bargaining game was that it enabled agents to collaborate and agree on a solution that benefited all parties rather than focusing on individual objectives. This model is particularly notable for incorporating agent preferences directly into the optimization process, offering a cooperative alternative to centralized scheduling.

A key trend in optimization methods for task scheduling is the integration of optimization with other techniques, like fuzzy logic, game theory, and risk-aware modeling, to ad-

dress uncertainty, heterogeneity, and scalability in real-world applications. These methods excel in generating high-quality solutions and are well-suited for small- to medium-scale problems where system parameters are well-defined. However, their main disadvantage lies in computational complexity; as problem size or dynamism increases, exact optimization becomes less tractable without decomposition, pruning, or approximation strategies. Despite this, their rigor and clarity in representing system behavior make them a valuable tool for high-stakes or safety-critical scheduling scenarios.

5.7. Approaches Including Path Planning and/or Obstacle/Collision Avoidance

Path planning and collision avoidance are critical components in the operation of mobile robots, especially in dynamic and complex environments such as warehouses or factories. Path planning involves determining the most efficient route for the robot to reach its destination, while collision avoidance ensures the robot avoids obstacles and other robots during navigation. Both techniques are essential for ensuring safety, efficiency, and task completion without interruption.

Efficient task scheduling between robots and simultaneous path planning are essential to avoid conflicts and optimize task execution time. Table 11 presents the records included in this review that incorporate path planning and/or collision avoidance. Only 16 out of 71 papers address these topics.

Table 11. Analysis of studies incorporating path planning and/or obstacle avoidance techniques.

Reference	Path Planning	Obstacle Avoidance
[19]	X	
[20]	X	X
[30]		X
[32]	X	X
[34]	X	X
[36]	X	X
[37]		X
[43]	X	X
[48]	X	
[51]	X	X
[52]	X	X
[55]	X	
[66]	X	
[67]	X	
[72]	X	
[84]	X	

Zhang et al. [20] presented a deterministic scheduling algorithm that assigned orders to the nearest available AGV within a specified range based on a rule-based approach. The deterministic routing algorithm employed a constrained BFS to plan collision-free paths, considering predicted future grid occupancy. Additionally, path planning was integrated, where the routing algorithm used a myopic requesting policy, occupying one grid unit at a time and ensuring collision avoidance. In the case of conflicts, priority was given to AGVs with earlier request times. Chung [30] implemented collision avoidance mechanisms through waiting and retreating strategies, which were managed by the scheduling procedure and the time-location graph. This model accounted for the continuous two-dimensional space (time and location) of the movements of the AGVs, ensuring that conflicts were resolved by adjusting the AGV paths and schedules to maintain safe and efficient operations. Pradhan et al. [32] incorporated motion coordination methods for groups of AGVs, aiming to respect their velocity and acceleration limitations while generating ap-

appropriate velocity profiles to ensure they followed their intended paths. Collision avoidance was achieved through these constraints during coordinated movement. The approach utilized the Potential Field Method and GA-Fuzzy algorithms for path planning. Additionally, it explored various cooperation strategies to manage potential collision scenarios, where AGVs could choose to either maintain their current path, modify both of their paths, or have one AGV sacrifice its motion. Game theory was applied to resolve coordination issues by constructing a payoff matrix based on cooperation and non-cooperation strategies. The Maximin and Minimax methods were then used to select the most appropriate strategy in coordination scenarios involving multiple AGVs. Chen and Liu [19] employed the A* algorithm and Rapidly-exploring Random Trees (RRT) for path planning. The A* algorithm was used for optimal path finding on a predefined grid, while RRT was employed to efficiently explore large, continuous spaces and generate feasible paths for the AGVs. Liu et al. [34] introduced a Conflict-Free Route Planning (CFRP) model aimed at minimizing travel time between origin and destination points for AMRs. The DijkstraTW algorithm was utilized to solve the CFRP problem, optimizing routes while considering reserved time windows for each AMR. Collision avoidance was a key element of the CFRP in the paper, ensuring that AMRs did not collide during their routes. The DijkstraTW algorithm incorporated obstacle avoidance and considered reserved time windows for links, during which only certain vehicles could occupy the route to prevent temporal conflicts. Additionally, it implemented waiting and route-changing strategies to address node conflicts and encounters between AMRs, ensuring safe and efficient path planning. Hu et al. [36] proposed a hierarchical planning method for dynamic scheduling of AGVs, with a path expert database generated using an improved A* algorithm to store optimal paths. Collision avoidance was handled by adjusting AGV paths by using waiting and detour strategies to prevent conflicts. A stage conflict avoidance method was demonstrated by Lu et al. [37], considering four AGV working stages: traveling to storage, transporting the pod to the workstation, returning to storage, and parking after task completion. Agrawal et al. [43] designed a reward function to optimize throughput and safety. The function provided rewards for completing machine jobs, with increasing rewards for subsequent machines to encourage efficiency. Penalties were given for movement actions to promote the shortest path navigation. Additionally, penalties were applied for collisions with walls, barriers, humans, other robots, or approaching processing machines, ensuring safety. Collision avoidance was learned through sensors, enabling the robot to navigate safely. Mayer et al. [48] employed the A* algorithm for path planning to calculate transport times. However, it did not address deadlock avoidance, leaving potential conflicts unresolved during navigation. The work by Wang et al. [51] presented a scheduling model that considered the number of depots and AGV battery consumption, incorporating a map complexity coefficient to account for the distance between workstations, obstacle area, distribution, and shape. The path planning used a decoupled prioritized algorithm to optimize AGV routes and ensure efficient navigation. Saidi-Mehrabad et al. [52] used an ACO, where the pheromone-update mechanism reinforced routes that led AGVs to required machines and reduced job completion times. More favorable routes, which minimized completion times and guided AGVs toward their goals, accumulated more pheromones, encouraging future AGVs to follow the same paths. Vivaldini et al. [55] and Zhao et al. [72] used Dijkstra's algorithm in their work for path planning, while Dou et al. [66] used RL, and Yi and Luo [67] used the A* algorithm. Lin et al. [84] proposed a Parallel Conflict-Based Search (PCBS) algorithm. PCBS utilized a thread pool at the upper level to allocate individual AGV path planning tasks to different threads. The lower level employed STA* for path planning, with communication between the upper and lower levels occurring through shared memory.

5.8. Experiments

This section analyzes the experimental methodologies and results from the 71 records included in this review, focusing on key factors such as the number of robots, evaluation metrics, and tested scenarios. By examining these aspects, we aim to identify trends, strengths, and gaps in the current research on task scheduling for mobile robots. This analysis provides a deeper understanding of how different scheduling strategies are tested and evaluated and highlights areas where future studies can further contribute to the field.

5.8.1. Distribution of Robots Across Included Records

During the data extraction phase, one of the key aspects analyzed was the maximum number of robots used in each study. A limitation identified in this review is that some studies did not specify the number of robots involved in their experiments. However, as shown in Table 12, the studies are categorized into three groups based on the number of robots: 2 to 5 robots, 6 to 10 robots, and more than 10 robots.

Table 12. Grouping of included records by number of robots used.

2–5 Robots	6–10 Robots	>10 Robots
[3,19,26,27,29–31,33,35,39,40,42–45,51–54,56–59,63–65,67,70,73–76,80,81,86]	[2,20,25,28,34,48,50,55,66,72,77,85]	[32,36–38,46,47,60,69,78,82,84,87]

Table 12 categorizes studies based on the number of robots used, revealing interesting trends in the field. The majority of studies (Group of 2 to 5 robots) focus on smaller-scale experiments due to feasibility, cost, simpler experimental setups, and simpler approaches. A moderate number of studies (Group of 6 to 10 robots) represent mid-range systems, offering a balance between complexity and scalability. In contrast, studies involving more than 10 robots are less common, which may be partially attributed to challenges such as coordination, resource limitations, and increased system complexity. This observation suggests a potential research gap in the mid-sized and large-scale fleet range, where further investigation could provide valuable insights into effective scaling strategies. Overall, the field appears to be evolving from smaller, proof-of-concept studies toward more complex and scalable systems, although additional research on medium-sized fleets could help bridge the gap between small- and large-scale implementations.

5.8.2. Small-Scale Systems Experiments

Small-scale systems (2–5 robots) account for all real-world experimental studies in the reviewed literature. These setups are primarily used to validate proof-of-concept algorithms in controlled environments, typically involving simple layouts, limited task diversity, and a small number of agents. While these conditions facilitate focused evaluation and repeatability, they often fall short of capturing the complexities present in larger, real-world deployments. Simulation environments are frequently employed to extend the scalability of these algorithms beyond what is feasible with physical systems. Table 13 organizes the experiments conducted in small-scale systems according to whether they were performed in real or simulated environments and according to the simulator used in each case.

Table 13. Small-scale systems experiments according to test type and simulator.

Real	MATLAB	Gurobi	CPLEX	Flexsim	Other
[33,39,45,56,67]	[19,27,31,42,44,51, 52,58,64,65,73,76, 80,81]	[63,68]	[3,29,59]	[53]	[26,30,35,40,41,43, 54,57,70,74,75,86]

Real-world experiments

Luo et al. [33] tested their ChEA algorithm in two scenarios. The first used a 6×6 instance of the Mousavi example [53] (six jobs and six machines), with 10 runs of 20 iterations each and a population size of 30. The second experiment used data from Bilge [91], which included four machines and different AGV and job-task combinations based on geographical layouts. The results were compared against evolutionary algorithms like MAS, GAHA, and FMAS. Zhang et al. [39] evaluated their DRL-based algorithm through both simulations and a real-world case study. The simulations involved 500 dynamic tasks, with release times following a Poisson distribution, and the real-world case was conducted in a production logistics system where AGVs operated on planned paths with speed adjustments and collision avoidance. Their method was compared to traditional scheduling approaches like MIWT-, MIEC-, and DQN-based methods. Gu et al. [45] built an experimental platform with physical entities such as AS/RS, AGVs, and robots connected via Wi-Fi. The system's performance was tested using a bio-inspired swarm optimization algorithm (BSOA), comparing it to static (HA) and dynamic (MAS) scheduling approaches in a setting with two AGVs and four machines. Mudrova and Hawes [56] validated their scheduler in simulated and real-world environments using mobile robots performing assistance tasks. The scheduler, implemented in C++ and distributed as a ROS package, showed improved performance compared to existing scheduling algorithms. Yi and Luo [67] tested their approach in a real robotic job shop (RJS) system with multiple machines, robotic arms, and AGVs. Their method, compared to a genetic algorithm (GA), demonstrated scalability and effectiveness for varying job loads.

These examples illustrate that while real-world testing provides valuable insights, it remains largely restricted to small-scale systems due to complexity, cost, and hardware limitations. No studies involving medium- or large-scale real-world systems were found in the reviewed records.

MATLAB simulation experiments

Samsuria et al. [27] used MATLAB to test their scheduling algorithms on benchmark datasets, including the EX44 instance from Bilge and Ulusoy [91] (four machines, with 19 operations across five jobs) and the 'la01' dataset from Lawrence [92] (10 jobs $\times$ 5 operations). Their approach was benchmarked against a genetic algorithm (GA) and a hybrid GA-Simulated Annealing method (HGASA), as well as other Bilge instances. Zhang et al. [31] used MATLAB to test their integrated AGV scheduling model on datasets from Brandimarte [93], including MK11–MK15 (middle-scale: 20 jobs, with 100–200 operations) and MK21–MK25 (large-scale: 50 jobs, with 250–500 operations). Processing times followed a uniform distribution. They also measured AGV energy consumption using a battery safety factor of 0.05 and compared the results with a standard GA. Chen and Liu [19] evaluated their decentralized method in MATLAB using Monte Carlo simulations. The scenarios involved 18 to 30 tasks with robots' initial positions randomized within a $100 \text{ m} \times 50 \text{ m}$ area. Robot speed was set at 2 m/s, and static obstacles were not considered. They compared their algorithm to CBBA and a centralized RHTA (C-RHTA). Xiao et al. [42] tested their improved Dynamic Particle Swarm Optimization (DPSO) algorithm using 10 examples from the PSPLIB library (J10, J12, J16, J18). They evaluated adaptability by simulating priority changes and increasing task load (including new workstations and AGVs). DPSO was reapplied in each scenario to re-optimize scheduling, and the results were benchmarked against PSO. Qu et al. [44] built a Jobset20-based case study including four machines, four tools,

and 10 parts with 24 operations. Mobile robots were responsible for inter-machine transport. A hybrid GA (population: 80; crossover: 0.9; mutation: 0.2) generated schedules within 24.23 s. The results included both Gantt charts and spatiotemporal trajectories of mobile robots. Wang et al. [51] used MATLAB to simulate three 2D workshop layouts: chaotic, linear, and obstacle-interrupted. The layouts involved 11–15 workstations, with four–five AGVs and scenarios with both single and multiple depots. Their method, incorporating a map complexity coefficient, outperformed the base model in layout-sensitive scheduling performance. Saidi-Mehrabadi et al. [52] combined GAMS and MATLAB for simulations involving two jobs, two AGVs, and three machines across 13 test problems. MATLAB was used to overcome the limitations of GAMS in handling large or dynamic configurations. Their Ant Colony Optimization method was compared with heuristic baselines. Al-Momani and Al-Aubidv [58] evaluated a fuzzy-based multi-robot scheduling system involving four programmable machines, a load/unload station, a charging station, and three mobile robots. Performance was benchmarked against Bilge [91], focusing on cost and time reduction. Samsuria et al. [64] ran three test cases in MATLAB (small and large scale) using Bilge and Ulusoy benchmarks. They compared the Standard GA (SGA), Particle Swarm Optimization (PSO), Static Time Window (STW), and multiple hybrid/modified GAs, including UGA, AGA, RGA, and IGA. Additional methods like Binary PSO, TS, ALS, HGA, MILP, SLSVNS, SOSA, and SBGA were also evaluated. Li et al. [65] tested a PSO-based scheduling system on a 50×28 grid simulation of a warehouse with three robots and one sorting station. Four task sets (12, 18, 16, and 9 tasks) were processed using PSO with 50 particles and 200 iterations. The model used an inertia weight of 0.7 and acceleration coefficients of 2. The results were presented in Gantt charts and compared against GA. Sun et al. [73] tested SSA and chaotic-enhanced SSA (CSSA) in MATLAB. The simulations used three and five AGVs, with each test repeated 20 times. The parameters included 100 iterations, a population of 100, and calibrated probabilities for crossover, mutation, and learning. CSSA consistently outperformed SSA in convergence and robustness. Abidi et al. [76] evaluated a metaheuristic method in MATLAB using 25 iterations and a population of 10. The method was tested against PSO, GWO, WOA, and LA, each combined with classifiers like SVM, Neural Networks (NNs), Deep Belief Networks (DBN), and Fuzzy Deep Belief Networks (FDBN). Pan et al. [80] tested Learning-based Multi-objective Evolutionary Optimizer (LMEO) using small-scale FJSPT1–10 instances and MK01–10 for medium/large-scale benchmarks. The number of AGVs was scaled with problem size, and machine coordinates were randomly assigned. Transportation time was computed using Euclidean distance. The results were evaluated using Relative Percentage Deviation (RPD) of Cmax. Wang and Xin [81] simulated a system with 10 jobs, three stages, and three AGVs, with four machines per stage. A genetic algorithm was applied to balance makespan and load distribution among AGVs.

Gurobi simulation experiments

Atik et al. [63] used the HydraOne AMR platform, integrating Jetson AGX and RealSense cameras with HydraNet for navigation and MobileNet-SSD for object detection. They simulated small, medium, and large instances, varying the number of AMRs, tasks, stations, and maintenance periods. Gurobi solved both an MINLP baseline and the LP version of their proposed MTC algorithm. Performance was compared against three baselines: RMA, RM, and MINLP, under varying maintenance requirements. Gurobi's academic license ensured reproducibility. Leet et al. [68] evaluated their approach with two real industrial scenarios: a Kiva (now Amazon Robotics) automated warehouse with two maps, WAREHOUSE1 (280 shelves and four stations), WAREHOUSE2 (240 shelves and 10 stations), and a package sorting center with 28 chutes and four bins (SORTINGCENTER map). The robot used in the simulations was a tricycle model. CCMP implementation was carried out using Python and the Gurobi solver to generate traffic plans. A 2 min time limit was

set for planners to start robot movements, with a total execution time defined as 3.6 times the number of products in the workload (in minutes).

CPLEX simulation experiments

Han et al. [29] tested their dual-resource scheduling method on 20 instances using CPLEX. Yao et al. [3] evaluated their MILP model on 82 benchmark instances divided by task-to-machine ratios, with representative examples like EX74 and EX740, and implemented the model in C++ using CPLEX. The results were compared to three existing MILP models. Dang et al. [59] used CPLEX Optimization Studio to solve an MIP model built in OPL, testing 21 uniformly randomized instances.

Flexsim simulation experiments

To validate the model, Mousavi et al. [53] used two numerical examples. The first involved six jobs processed on six machines, with each job consisting of between two and five operations. The second example involved 15 jobs processed on 10 machines, with each job having between one and five operations. These were followed by experiments conducted in a simulated FMS environment using FlexSim software. The paper compared the performance of the proposed hybrid algorithm with those of the GA and PSO algorithms.

Other simulation experiments

Avhad et al. [26] tested a Swarm Production System (SPS) in a Software-in-the-Loop (SiL) simulation using Visual Components. A case study with 3 Transportation Robots (TRs) and 10 Workstation Robots (WRs) demonstrated the Swarm Manager's ability to coordinate planning, scheduling, and control. The impact of three topologies—Linear, Matrix, and Swarm—on performance was evaluated. Integration with the simulation was achieved via OPC-UA. The authors simulated a display manufacturing facility using real-world data from two DRIS units (N1 and N2) in South Korea, incorporating robot velocity, zone lengths, and FIFO job selection rules. The specific simulator was not mentioned. Mareddy et al. [40] evaluated the adapted SOSA algorithm in a Flexible Manufacturing System (FMS) setup with four CNCs, two AGVs, a central tool magazine, and four layout configurations. A total of 10 job sets were tested with varying transport-to-processing time ratios. A real-world case from an automotive supplier was also included and compared with the Jaya algorithm. Durst et al. [35] modeled a simplified job shop in SimPy, integrating a Deep Reinforcement Learning-based AGV scheduler using TensorFlow. The setup had three AGVs, sources/sinks, and machines with buffers. The study compared FIFO, STD, NJF, LWT, SRPT, and EDD dispatching rules. Geng et al. [74] simulated a machining workshop with 13 IoT-enabled stations. Real-time data guided a control center running the MAPPO-GC algorithm. Its seven hyperparameters were optimized using Optuna and manual tuning. Xiaoting et al. [75] tested DQNG on 10 FJSP benchmark instances, each repeated 10 times. Comparisons were made with ILS, TS, GA, and BRKGA. Agrawal et al. [43] used Unity ML Agents Toolkit to validate agent adaptability under dynamic conditions such as breakdowns and processing delays in a virtual factory. Baruwa and Piera [54] used the TCPN tool TIMSPAT with ALS and A* algorithms coded in C++ to test 82 problem instances from Bilge and Ulusoy [91]. Instances were grouped by ratio size (>0.25 and <0.25), and model performance (MCSS vs. VCSS) was evaluated. Kim et al. [41] evaluated their proposed algorithm in Siemens Plant Simulation, using three baselines: (i) Minimum Time Until Processing, (ii) Shortest Traveling Time, and (iii) Random. The battery charging of the AMRs followed a threshold-based rule with a CONWIP input control policy. Performance was assessed using the average cycle time of car bodies. The small-scale layout included 19 workstations and one charging station, with each workstation performing 4–14 operations. The layout and processing data were based on the specifications of an industry partner. Dang et al. [57] evaluated their process on a set of 82 benchmark instances. The instances were divided into two groups based on the ratio of average traveling time to average processing time (t/p). This allows for the analysis of the algorithm's performance under different scenarios. Cechinel

and De Pieri [70] evaluated their approach in a simulated environment created in the Gazebo simulator. This simulated environment represented a warehouse measuring approximately 1500 m², with 8 parking stations, 5 delivery locations, and 192 pickup locations. A grid map was used to represent the environment of the robots' navigation system. The authors encountered navigation problems due to the detection of other robots as obstacles leading to task failures.

Across the literature, a strong preference for small-scale systems is evident, particularly in real-world implementations, due to constraints in cost, complexity, and scalability. Simulations, especially in MATLAB and optimization environments like Gurobi and CPLEX, dominate experimental setups and allow for broader scalability testing and parameter exploration. Additionally, while many approaches show promising performance improvements, comparative consistency across platforms and standardized benchmarks is lacking. Future research should address scalability in physical deployments, enhance reproducibility, and promote the use of standardized, open-access benchmarks across both real and simulated environments.

5.8.3. Medium-Scale Systems Experiments

Medium-scale systems (6–10 robots) are predominantly evaluated through simulation studies, offering a balance between realism and scalability. These setups typically use standardized benchmarks or custom scenarios to test optimization algorithms under controlled yet varied conditions. Common platforms include MATLAB and Python with Gurobi, along with proprietary or competition-based simulators. Experiments often explore scheduling complexity, AGV coordination, and layout-specific constraints while analyzing performance metrics such as makespan, energy consumption, and system robustness. These studies bridge the gap between small-scale physical implementations and theoretical large-scale models. Table 14 organizes the experiments conducted in medium-scale systems according to simulated environments and the simulator used in each case.

Table 14. Medium-scale system experiments according to the simulator used.

MATLAB	Gurobi	Other
[2,28,34,77]	[25]	[20,48,50,55,66,72,85]

MATLAB simulation experiments

Samsuria et al. [27] and Xu et al. [28] used benchmark datasets from Brandimarte [93] and Bilge and Ulusoy [91], implementing their optimization algorithms in MATLAB. He et al. [2] evaluated 39 benchmark instances across different scales (4 × 4, 6 × 6, 10 × 10, 20 × 5, 20 × 10, and 30 × 10), varying AGV numbers from 2 to mm. The shop floor had a U-shaped layout, and AGVs moved clockwise. Experiments defined processing times, speeds, energy consumption, deterioration rates, and due dates using uniform distributions. Their method was compared with NSGA-II, NSGA-III, MOEA/D, and HACO. Liu et al. [34] tested their SLGA-D algorithm on seven examples (EX1–EX7), comparing it to GA-D, AGA-D, and HPSO-D in terms of solution quality, stability, and runtime. Additional experiments tested dispatching strategies (DSDS vs. FDS) using EX8 and EX9 and analyzed makespan sensitivity with varying AMR counts (1–10 jobs on four machines, EX10) and job counts (1–15 jobs with four machines and four AMRs, EX11). Krishnamoorthy et al. [77] used a spinning mill case study to validate their model under two machine configurations (8 and 12 machines). MATLAB simulations were run 50 times with a population of 100 over 100 iterations. The results were benchmarked against existing models, including improved PSO, GA + PSO hybrids, harmony search-based scheduling, and GA + ACO methods tailored for AGV environments.

Gurobi simulation experiments

The study by Qin et al. [25] employed Python with Gurobi to evaluate experimental instance groups of small, medium, and large scales. The workload balance ratios were varied from 0 to 1 across 11 different scenarios to analyze their impact on performance. The study also examined the effect of order characteristics by considering different order numbers. Large-scale scenarios, including LI1, LI2, LI3, and LI4, were compared to assess the system's efficiency. The analysis included configurations with 2 to 10 robots, focusing on how varying robot numbers and workload distributions influenced overall performance.

Other simulation experiments

Zhang et al. [20] tested their method in three warehouse scenarios from a competition, varying grid sizes (18×18 and 20×20), AGV counts (8 and 10), and berth/obstacle layouts. Order arrival rates were set at 70 and 120 orders. Additional parameters included AGV speed, loading/unloading time, simulation duration, and a penalty term in the ACT. The code was published, but the simulator was not specified. Mayer et al. [48] simulated six parameter sets to study the impact of AGV count, buffer sizes, task rescheduling, and task release mechanisms (random vs. fixed) across different WIP levels. The metrics included makespan variability, deadlock occurrence, and performance against an idealized optimal solution. Khosiawan et al. [50] evaluated their method in both lab- and industrial-scale environments with varying task sizes and precedence constraints. Agent teams (three UAVs + two or three AGVs) were used to test coordination effectiveness. The results were compared with standard Differential Evolution (DE) and Particle Swarm Optimization (PSO) algorithms. Vivaldini et al. [55] simulated two warehouse layouts: Layout 1, with 5 shelves, and Layout 2, with 10. Each included three docks and six depots, with AGVs navigating bidirectional paths. Two orders of 27 unloading tasks (three trucks $\times$ nine pallets) were tested per layout, comparing the results with Nearest Neighbor and Tabu Search algorithms. Dou et al. [66] tested their approach using eight mobile robots in a smart grid warehouse environment. It includes path planning but no coordination between robots, so the system fails when collisions happen. Zhao et al. [72] tested their approach with 10 robots, and just like the previous authors [66], the approach only includes path planning, making the solution less scalable. The system developed by Ho et al. [85] was simulated for an e-commerce scenario involving three geographically distributed warehouses with heterogeneous configurations, including varying block layouts and robot speeds. Task arrivals followed a Poisson distribution (1–6 tasks/min), and task locations were uniformly distributed across rack blocks. Robot speeds varied per time slot using uniform distributions, differing by warehouse. The Neural Networks were trained using PyTorch.

Simulation-based experiments in medium-scale systems provide valuable insights into the performance and adaptability of AGV and AMR scheduling methods. While most studies demonstrate comparative advantages over baseline algorithms, the reliance on synthetic benchmarks and the absence of standardized platforms limit cross-study comparability. Moreover, the lack of real-world physical validation in this scale category indicates an area for future development.

5.8.4. Large-Scale Systems Experiments

Table 15 presents a classification of studies that evaluated robotic and scheduling systems in large-scale systems (>10 robots), distinguishing them by the simulation tools used, including MATLAB, Gurobi, FlexSim, and other custom or hybrid frameworks. These experiments reflect a growing research focus on scaling up task planning, scheduling, and coordination mechanisms in complex warehouse and logistics domains, often featuring dynamic, heterogeneous agents and environments.

Table 15. Large-scale systems experiments according to the simulator used.

MATLAB	Gurobi	Flexsim	Other
[32,60]	[38,69]	[36]	[37,46,47,78,82,84,87]

MATLAB simulation experiments

Pradhan et al. [32] considered a dynamic 2D environment in their problem formulation, and simulations were conducted in MATLAB to validate their proposed method. These simulations used accurate models of mobile robots performing various tasks with different robot configurations. To present the results of the queuing model, the MAPLE 18 and MATHEMATICA 9 software packages were used for further analysis and illustration. Bakshi et al. [60] tested the QCR algorithm on a simple 10-node symmetric TSP to analyze its performance. The results were compared to the orthogonal relaxation method and the case without regularization ($\alpha = 0$). In this experiment, the QCR algorithm was evaluated before the weight matching step, where the weight of each edge in the solution was represented in the STTS matrix.

Gurobi simulation experiments

Fu et al. [38] evaluated the CTAS framework using two simulated scenarios. The first, a Capture the Flag Game, featured heterogeneous agents (differing in speed, health, and ammunition) and compared the task assignment of CTAS with STRATA and random assignment. The second scenario, Robotic Services During a Pandemic, involved subtasks like delivery, disinfection, testing, and treatment, modeled across a city using the M3500 road map and a viral exposure-based cost map. Agent capabilities and task demands followed Gaussian distributions. CTAS components (task assignment, scheduling, and generality) were assessed with Gurobi used for optimization. Boysen et al. [69] conducted experiments on three datasets: ABC (skewed SKU demand), EQ (uniform SKU distribution), and real-world e-commerce data from South Asia. The instances varied in terms of robot and order counts. Gurobi (300 s time limit) was used for solving. The tested methods included rule-based (RULE), random (RND), and decomposition-based approaches (DEC), combining MIPs and heuristics for subproblems: DEC(MIP,MIP) and DEC(HEU,HEU). A single-step MSA variant was also tested against the full two-step MSA (60 scenarios, 120-step horizon) to assess the benefit of the full approach.

Flexsim simulation experiments

To evaluate the proposed dynamic scheduling method, Hu et al. [36] conducted computational experiments in a dynamic scenario involving varying fleet sizes and task numbers. They used warehouse production data from Changsha, China, with a layout that included 12 buffer area depots, 12 shop depots, 15 automatic vertical warehouse depots, and five charging stations. A distance matrix was employed to compute AGV travel times, and a FlexSim-based digital simulation system was utilized to analyze AGV operations. The dynamic scheduling involved 60 tasks, with new tasks added at specific times, necessitating rescheduling. Gantt charts illustrated the initial and updated schedules, showing task execution and pre-emption during interruptions. The HDSTA algorithm, integrated with the adaptive large neighborhood search (HALNS) algorithm, was compared to the preplanning algorithm (PPA).

Other simulation experiments

Lu et al. [37] simulated a Robotic Mobile Fulfillment System (RMFS) with 20 storage blocks in a 2×5 layout. Picking tasks were randomly generated, with picking and lifting/releasing times of 8 s and 3 s per pod. AGVs moved bidirectionally at 100 cm/s, except near the picking table. A Simulated Annealing (SA) algorithm was applied with defined temperature parameters and cooling rates. Experiments varied the number of AGVs (5, 8, 10, and 14), workstations (two, three, and five), and task sizes (15, 30, 50,

and 100). A class-based storage scenario was also tested, where 80% of picking demand came from specific zones. The results were compared with RAR, CAR, OTAR, TS, and SA-I. Bolu and Korcak [46] developed a digital twin using a Warehouse Execution System (WES) simulation that incorporated real-world factors like robot collisions and energy usage. Scenarios tested included varying order sizes, robot workload balancing, and SKU configurations. Riazi and Lennartson [47] evaluated their algorithm in Python using Z3 and IBM ILOG CP solvers, with parallelization disabled for fairness. The tests included 8–12 tasks and 4–12 AGVs, simulating 45 min of Volvo Cars production, with a 40 min scheduling horizon and a 3600 s time limit. Dang et al. [87] implemented a DES model of a COPS warehouse in Python using SimPy. Three scenarios were tested: (1) AMR and picker coordination, tracking congestion and disturbances; (2) dynamic picker assignment using AMR availability; and (3) AMR overtaking, where distances for safe passing were computed based on warehouse layout. Key components (pickruns, AMRs, pickers, and racks) were initialized before simulation execution. Lin et al. [84] used a small-scale cloud-edge setup with two servers and two PCs. The tests evaluated task scheduling (local vs. cloud PVNS), path planning (PCBS with varying AGVs), and overall system performance in a 20×20 warehouse with 12 AGVs and 40 tasks. The study by Feo-Flushing et al. [82] involved simulations on 2D grid environments of various sizes (5×5 to 20×20) with heterogeneous agent teams (4–20 agents across four performance classes).

A notable trend across the reviewed studies is the increasing use of heterogeneous agent models, realistic spatial layouts, and multi-objective optimization frameworks to better simulate industrial-scale environments. Tools like Gurobi and custom simulations in Python or C++ were favored for flexibility and computational power, while FlexSim and MATLAB provided visual analysis and quick prototyping.

However, several gaps persist. Many simulations lack real-time constraints or omit inter-agent communication failures, which are critical in operational settings. Furthermore, few studies integrate energy consumption, maintenance scheduling, or unexpected disruptions, which are all vital for deployment in real-world warehouse and production systems.

Looking forward, future research should aim to develop standardized benchmarks for large-scale systems, incorporate hybrid physical-digital experimentation (e.g., digital twins), and address scalability in more detail. Integrating reinforcement learning with formal optimization, as well as simulating uncertainty in demand and supply, also represents promising directions to better bridge simulation and deployment in real-world automated systems.

5.8.5. Evaluation Metrics

During the data extraction phase in Section 3.4, one of the items defined for data extraction was #DE6, which refers to the evaluation metrics used in each paper to assess the performance and efficiency of the systems under study. To make the visualization of this analysis clearer, a word cloud, as shown in Figure 7, was created in Python, highlighting the most commonly mentioned evaluation metrics across the reviewed literature.

The *makespan*, defined as the total time to complete all tasks, emerged as the most frequently evaluated metric, appearing 51 times in the records. This is expected, as makespan is a critical performance measure in many optimization and scheduling problems. Other frequently mentioned metrics include *utilization of robots*, which appeared 11 times, and *computation time*, appearing 33 times. The *generated schedule* is also a strong evaluation metric since many studies include the Gantt chart of the generated schedule in their results for better visualization. These metrics are central to understanding the operational efficiency of systems, especially in contexts involving automation and task scheduling.

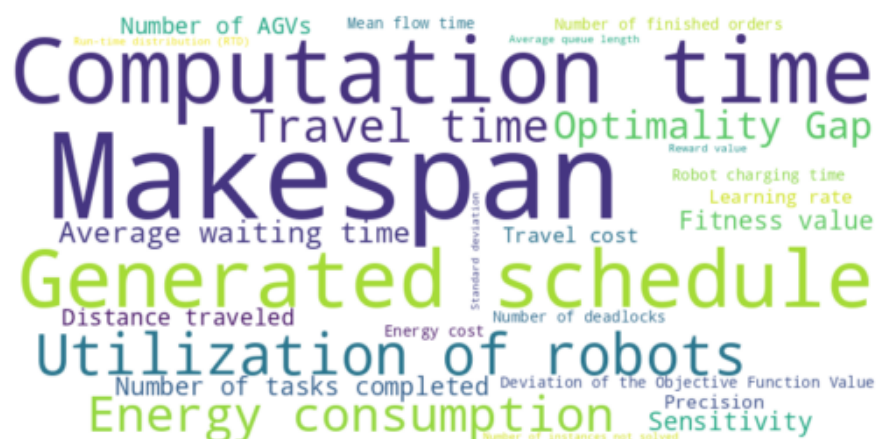


Figure 7. Word Cloud of evaluation metrics used in the reviewed papers.

Additional important metrics include *energy consumption* (eight mentions), as many included works aim for energy efficiency, and *travel time* (seven mentions), related to works including path planning with the aim for the shortest path. Both of these metrics are crucial in evaluating the sustainability and efficiency of automated systems. In relation to operational performance, *average waiting time* and *number of tasks completed* were also frequently assessed. These metrics provide insights into system responsiveness and task throughput, which are critical in evaluating the effectiveness of scheduling algorithms.

In terms of solution quality, *optimality gap*, representing the deviation from the best possible solution, appeared seven times, reflecting the importance of solution accuracy in optimization tasks. *fitness value*, another key measure of solution quality, was mentioned 6 times.

Other notable metrics include *number of deadlocks*, which indicate potential system inefficiencies, and *robot charging time*, relevant in the context of systems that rely on battery-powered robots. On the statistical side, metrics like *precision*, *sensitivity*, and *mean flow time* were discussed but less frequently.

This analysis through a word cloud provides a clear picture of the metrics that are most relevant in the context of the reviewed works, offering a snapshot of the areas of performance that researchers focus on when evaluating system effectiveness.

6. Limitations of the Study

This systematic review follows a structured methodology to select and analyze papers related to task scheduling with mobile robots. However, despite using a systematic approach, the study has some limitations.

One limitation of this review is related to the study population, mobile robots, as defined in Section 3.2, which focuses on a specific group of vehicles. This constraint may limit the included records, potentially excluding other vehicle types that could also be relevant to the task scheduling problem. Furthermore, as initially mentioned, the study only considers terrestrial mobile robots, excluding UAVs and other types, because the scheduling and planning challenges in the terrestrial domain differ significantly from those in aerial or underwater systems. Factors like ground-level navigation constraints, energy consumption models, task execution environments, and interaction with obstacles or terrain are context-specific and distinct from those faced by aerial robots. However, by focusing the review on terrestrial mobile robots, it is possible to explore a broader range of task scheduling problems, such as JSSP, where more complex algorithms are implemented and needed. This is due to the unpredictable nature of factors like machine failures, obstacles,

and human interference, issues that are not typically studied in other vehicle types, as they are not applied in industrial or hospital environments.

The search strategy used in this review may present a potential limitation. For example, if a paper does not include any of the keywords from the search query (e.g., mobile robots or task planning) in the abstract, title, or keywords, the methodology outlined in Section 3 might not identify it. However, the search query goes beyond task scheduling by including terms like task planning and schedul*, thereby expanding the scope. This broader approach is intended to minimize the risk of missing relevant records related to the topic under investigation.

In the quality assessment of the selection phase, this study only considers records that score 1.0/1.0 in QE3, where the question is whether the study includes multi-robot coordination or cooperation, thereby excluding single-robot methodologies. Single-robot methodologies are typically associated with solutions that are not scalable to more robots or complex scenarios, which is why they were excluded. However, by doing so, there is a risk of excluding potentially valuable papers that, despite focusing on single-robot systems, may offer scalable solutions.

Another limitation concerns the year range, from 2014 to 2024. This choice is explained in Section 3, but it inherently excludes earlier studies that may offer valuable insights into the foundational development of the field. Expanding the temporal scope in future reviews could provide a more comprehensive understanding of the evolution of task scheduling with mobile robots, complementing the findings from this review.

7. Conclusions and Future Directions

This paper presents a systematic literature review of task scheduling with mobile robots, selecting 71 relevant works that highlight the main strategies for achieving efficient task scheduling. It also discusses the experimental data and evaluation metrics commonly used to assess the performance of these approaches. The analysis categorizes the works based on robot configuration and communication architecture within fleets, revealing that most studies focus on homogeneous fleets and centralized architectures. The reviewed records were further categorized by the algorithms/approaches used (e.g., heuristic, meta-heuristic, optimization, hybrid, and machine learning), with metaheuristic and hybrid methods being the most prevalent. Given the complexity of task scheduling with mobile robots, these methods are particularly well-suited for managing such complexity. An emerging trend observed in the analysis is the growing use of machine learning techniques, driven by their adaptability.

Despite the increasing relevance of this topic, as highlighted in Section 3, it remains an area with considerable potential for further exploration and scaling. Notably, only 17 out of the 71 reviewed works address issues such as path planning and collision avoidance as complementary elements to task scheduling. These aspects are critical for making the systems scalable and adaptable to dynamic environments, indicating an important direction for future research.

The analysis also revealed that few studies incorporate large-scale fleets of robots due to the lack of robustness in existing solutions and the computational complexity involved. This presents an opportunity for future research to focus on developing more scalable and efficient solutions. Moreover, few studies have been tested in real-world environments, indicating that this research area is still in its early stages. Future research should integrate factors like machine and robot failures, along with other real-world challenges, to better address the complexities and unpredictability of actual operational settings.

Finally, this review is open to updates in future studies, as it follows a systematic review methodology based on the PRISMA statement. This methodology explicitly defines

the selection process, time frame, and research queries used to identify records from the literature. Future extensions of this paper could delve deeper into various robot topologies and provide a more thorough comparison of different methodologies.

Author Contributions: Conceptualization, C.R., P.C. and M.S.; data curation C.R.; investigation, C.R.; methodology, C.R.; supervision, P.C., M.S. and E.J.S.P.; validation, P.C., M.S. and E.J.S.P.; writing—original draft, C.R.; writing—review and editing, C.R., P.C., M.S. and E.J.S.P. All authors have read and agreed to the published version of the manuscript.

Funding: This work is financed by National Funds through the Portuguese funding agency, FCT-Fundação para a Ciência e a Tecnologia, within project LA/P/0063/2020. DOI 10.54499/LA/P/0063/2020—<https://doi.org/10.54499/LA/P/0063/2020>.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

References

1. Sahar, N.; Basile, A.; Yan, B. A Tightened Formulation for Job Shop Scheduling with Mobile Robots. In Proceedings of the 2024 IEEE 20th International Conference on Automation Science and Engineering (CASE), Bari, Italy, 28 August–1 September 2024; pp. 3237–3242. [\[CrossRef\]](#)
2. He, L.; Chiong, R.; Li, W. Energy-efficient open-shop scheduling with multiple automated guided vehicles and deteriorating jobs. *J. Ind. Inf. Integr.* **2022**, *30*, 100387. [\[CrossRef\]](#)
3. Yao, Y.J.; Liu, Q.H.; Li, X.Y.; Gao, L. A novel MILP model for job shop scheduling problem with mobile robots. *Robot. Comput. Integr. Manuf.* **2023**, *81*, 102506. [\[CrossRef\]](#)
4. Tejer, M.; Szczepanski, R.; Tarczewski, T. Robust and efficient task scheduling for robotics applications with reinforcement learning. *Eng. Appl. Artif. Intell.* **2024**, *127*, 107300. [\[CrossRef\]](#)
5. Moher, D.; Liberati, A.; Tetzlaff, J.; Altman, D.G.; Group, T.P. Preferred reporting items for systematic reviews and meta-analyses: The PRISMA statement. *PLoS Med.* **2009**, *6*, e1000097. [\[CrossRef\]](#) [\[PubMed\]](#)
6. Li, J. Comparative Study and Hybrid Modeling of Vehicle Routing Problem and Job Shop Scheduling Problem. In *Recent Trends in Decision Science and Management*; Wang, T.S., Ip, A., Tavana, M., Jain, V., Eds.; Advances in Intelligent Systems and Computing; Springer: Singapore, 2020; Volume 1142. [\[CrossRef\]](#)
7. Liaqait, R.; Hamid, S.; Warsi, S.; Khalid, A. A Critical Analysis of Job Shop Scheduling in Context of Industry 4.0. *Sustainability* **2021**, *13*, 7684. [\[CrossRef\]](#)
8. Xiong, H.; Shi, S.; Ren, D.; Hu, J. A Survey of Job Shop Scheduling Problem: The Types and Models. *Comput. Oper. Res.* **2022**, *142*, 105731. [\[CrossRef\]](#)
9. Quinton, F.; Grand, C.; Lesire, C. Market Approaches to the Multi-Robot Task Allocation Problem: A Survey. *J. Intell. Robot. Syst.* **2023**, *107*, 29. [\[CrossRef\]](#)
10. K A, A.; Udayan J, D.; Subramaniam, U. A Systematic Literature Review on Multi-Robot Task Allocation. *ACM Comput. Surv.* **2024**, *57*, 68. [\[CrossRef\]](#)
11. Freitas, V. Parsifal, 2014. Available online: <https://parsif.al/> (accessed on 24 March 2025).
12. NotebookLM. 2025. Available online: <https://notebooklm.google/> (accessed on 4 March 2025).
13. ACM Digital Library, n.d. Available online: <https://dl.acm.org/> (accessed on 24 October 2024).
14. Scopus. Scopus, n.d. Available online: <https://www.scopus.com/search/form.uri> (accessed on 24 October 2024).
15. IEEE Xplore. IEEE Xplore Digital Library, n.d. Available online: <https://ieeexplore.ieee.org/Xplore/home.jsp> (accessed on 24 October 2024).
16. Web of Science. Web of Science, n.d. Available online: <https://www.webofscience.com/wos/woscc/basic-search> (accessed on 24 October 2024).
17. van Eck, N.J.; Waltman, L. Software survey: VOSviewer, a computer program for bibliometric mapping. *Scientometrics* **2010**, *84*, 523–538. [\[CrossRef\]](#)
18. van Eck, N.J.; Waltman, L. Visualizing bibliometric networks. In *Measuring Scholarly Impact*; Ding, Y., Rousseau, R., Wolfram, D., Eds.; Springer: Cham, Switzerland, 2014; pp. 285–320. [\[CrossRef\]](#)
19. Chen, X.; Liu, S. Distributed Cooperative Task Planning for Autonomous Mobile Robots in Intralogistics. In Proceedings of the 9th 2023 International Conference on Control, Decision and Information Technologies, CoDIT 2023, Rome, Italy, 3–6 July 2023; pp. 870–875. [\[CrossRef\]](#)

20. Zhang, G.B.; Li, H.B.; Liu, X.T.; Peng, Y.J. Simulation Budget Allocation for Improving Scheduling and Routing of Automated Guided Vehicles in Warehouse Management. *J. Oper. Res. Soc. China* **2024**. [\[CrossRef\]](#)
21. Li, W.; Li, H.; Wang, Y.; Han, Y. Optimizing flexible job shop scheduling with automated guided vehicles using a multi-strategy-driven genetic algorithm. *Egypt. Inform. J.* **2024**, *25*, 100437. [\[CrossRef\]](#)
22. Liu, Z.J.; Sang, H.Y.; Zheng, C.Z.; Chi, H.; Gao, K.Z.; Han, Y.Y. An effective multi-restart iterated greedy algorithm for multi-AGVs dispatching problem in the matrix manufacturing workshop. *Expert Syst. Appl.* **2024**, *252*, 124223. [\[CrossRef\]](#)
23. Pang, H.; Zhen, L. Automated mobile robots routing and job assignment in automated factory. *Comput. Ind. Eng.* **2024**, *195*, 110420. [\[CrossRef\]](#)
24. Justkowiak, J.E.; Kovalyov, M.; Pesch, E. A dynamic programming algorithm for order picking in robotic mobile fulfillment systems. *Networks* **2024**, *84*, 481–490. [\[CrossRef\]](#)
25. Qin, Z.; Kang, Y.; Yang, P. Making better order fulfillment in multi-tote storage and retrieval autonomous mobile robot systems. *Transp. Res. Part E Logist. Transp. Rev.* **2024**, *192*, 103752. [\[CrossRef\]](#)
26. Avhad, A.; Arnarson, H.; Schou, C.; Madsen, O. Implementing Swarm Production System with Multi-Robot Simulation. *Procedia Comput. Sci.* **2024**, *232*, 934–945. [\[CrossRef\]](#)
27. Samsuria, E.; Mahmud, M.; Wahab, N.; Romdlony, M.; Abidin, M.; Buyamin, S. Solving an Integrated Job-Shop–Mobile Robot Scheduling Problem in Flexible Manufacturing System using Enhanced Genetic Algorithm Structure with Local Search Method. *Appl. Model. Simul.* **2024**, *8*, 225–238.
28. Xu, G.; Bao, Q.; Zhang, H. Multi-objective green scheduling of integrated flexible job shop and automated guided vehicles. *Eng. Appl. Artif. Intell.* **2023**, *126*, 106864. [\[CrossRef\]](#)
29. Han, X.; Cheng, W.; Meng, L.; Zhang, B.; Gao, K.; Zhang, C.; Duan, P. A dual population collaborative genetic algorithm for solving flexible job shop scheduling problem with AGV. *Swarm Evol. Comput.* **2024**, *86*, 101538. [\[CrossRef\]](#)
30. Chung, J. A simulation and scheduling method for analyzing the peak time capacity of the dual-robot in-line stocker. *J. Simul.* **2024**, *18*, 835–850. [\[CrossRef\]](#)
31. Zhang, L.; Yan, Y.; Hu, Y. Integrated Scheduling of Flexible Job Shop and Energy-Efficient Automated Guided Vehicles. In Proceedings of the 2023 8th IEEE International Conference on Advanced Robotics and Mechatronics, ICARM 2023, Sanya, China, 9–11 July 2023; pp. 493–498. [\[CrossRef\]](#)
32. Pradhan, B.; Goswami, V.; Barik, R.; Sahana, S. An integrated strategy-based game-theoretic model and decentralized queueing system for mobile multi-robot task coordination. *Decis. Anal. J.* **2023**, *7*, 100254. [\[CrossRef\]](#)
33. Luo, Y.; Zhang, Q.; Lin, L. A Cooperative Hybrid Evolutionary Algorithm for Flexible Scheduling with AGVs. In Proceedings of the ICSMD 2023—International Conference on Sensing, Measurement and Data Analytics in the Era of Artificial Intelligence, Proceedings, Xi'an, China, 2–4 November 2023. [\[CrossRef\]](#)
34. Liu, J.; Sun, B.; Li, G.; Chen, Y. An integrated scheduling approach considering dispatching strategy and conflict-free route of AMRs in flexible job shop. *Int. J. Adv. Manuf. Technol.* **2023**, *127*, 1979–2002. [\[CrossRef\]](#)
35. Durst, P.; Jia, X.; Li, L. Multi-Objective Optimization of AGV Real-Time Scheduling Based on Deep Reinforcement Learning. In Proceedings of the Chinese Control Conference (CCC), Tianjin, China, 24–26 July 2023; pp. 5535–5540. [\[CrossRef\]](#)
36. Hu, E.; He, J.; Shen, S. A dynamic integrated scheduling method based on hierarchical planning for heterogeneous AGV fleets in warehouses. *Front. Neurobot.* **2023**, *16*, 1053067. [\[CrossRef\]](#)
37. Lu, J.; Ren, C.; Shao, Y.; Zhu, J.; Lu, X. An automated guided vehicle conflict-free scheduling approach considering assignment rules in a robotic mobile fulfillment system. *Comput. Ind. Eng.* **2023**, *176*, 108932. [\[CrossRef\]](#)
38. Fu, B.; Smith, W.; Rizzo, D.; Castanier, M.; Ghaffari, M.; Barton, K. Robust Task Scheduling for Heterogeneous Robot Teams under Capability Uncertainty. *IEEE Trans. Robot.* **2023**, *39*, 1087–1105. [\[CrossRef\]](#)
39. Zhang, L.; Yan, Y.; Hu, Y. Deep reinforcement learning for dynamic scheduling of energy-efficient automated guided vehicles. *J. Intell. Manuf.* **2024**, *35*, 3875–3888. [\[CrossRef\]](#)
40. Mareddy, P.; Narapureddy, S.; Dwivedula, V.; Karanam, P. Optimum scheduling of machines, automated guided vehicles and tools without tool delay in a multi-machine flexible manufacturing system using symbiotic organisms search algorithm. *Concurr. Comput. Pract. Exp.* **2022**, *34*, e6950. [\[CrossRef\]](#)
41. Kim, M.S.; Oh, S.C.; Chang, E.; Lee, S.; Wells, J.; Arinez, J.; Jang, Y. A dynamic programming-based heuristic algorithm for a flexible job shop scheduling problem of a matrix system in automotive industry. In Proceedings of the IEEE International Conference on Automation Science and Engineering, Mexico City, Mexico, 20–24 August 2022; pp. 777–782. [\[CrossRef\]](#)
42. Xiao, X.; Pan, Y.; Lv, L.; Shi, Y. Scheduling multi-mode resource-constrained tasks of automated guided vehicles with an improved particle swarm optimization algorithm. *IET Collab. Intell. Manuf.* **2021**, *3*, 93–104. [\[CrossRef\]](#)
43. Agrawal, A.; Won, S.; Sharma, T.; Deshpande, M.; McComb, C. A multi-agent reinforcement learning framework for intelligent manufacturing with autonomous mobile robots. In *Proceedings of the Design Society*; Cambridge University Press: Cambridge, UK, 2021; Volume 1, pp. 161–170. [\[CrossRef\]](#)

44. Qu, S.; Hu, Y.; Ren, W.; Yang, X. Coordinative scheduling of the mobile robots and machines based on hybrid GA in flexible manufacturing systems. *Procedia CIRP* **2021**, *104*, 1005–1010. [\[CrossRef\]](#)
45. Gu, W.; Li, Y.; Zheng, K.; Yuan, M. A bio-inspired scheduling approach for machines and automated guided vehicles in flexible manufacturing system using hormone secretion principle. *Adv. Mech. Eng.* **2020**, *12*. [\[CrossRef\]](#)
46. Bolu, A.; Korkak, O. Adaptive Task Planning for Multi-Robot Smart Warehouse. *IEEE Access* **2021**, *9*, 27346–27358. [\[CrossRef\]](#)
47. Riazzi, S.; Lennartson, B. Using CP/SMT Solvers for Scheduling and Routing of AGVs. *IEEE Trans. Autom. Sci. Eng.* **2021**, *18*, 218–229. [\[CrossRef\]](#)
48. Mayer, S.; Hohme, N.; Gankin, D.; Endisch, C. Adaptive production control in a modular assembly system—Towards an agent-based approach. In Proceedings of the IEEE International Conference on Industrial Informatics (INDIN), Helsinki, Finland, 22–25 July 2019; pp. 45–52. [\[CrossRef\]](#)
49. Nabovati, H.; Haleh, H.; Vahdani, B. Fuzzy multi-objective optimization algorithms for solving multi-mode automated guided vehicles by considering machine break time and artificial neural network. *Neural Netw. World* **2018**, *28*, 255–283. [\[CrossRef\]](#)
50. Khosiawan, Y.; Khalfay, A.; Nielsen, I. Scheduling unmanned aerial vehicle and automated guided vehicle operations in an indoor manufacturing environment using differential evolution-fused particle swarm optimization. *Int. J. Adv. Robot. Syst.* **2018**, *15*. [\[CrossRef\]](#)
51. Wang, F.; Zhang, Y.; Su, Z. A novel scheduling method for automated guided vehicles in workshop environments. *Int. J. Adv. Robot. Syst.* **2019**, *16*. [\[CrossRef\]](#)
52. Saidi-Mehrabad, M.; Dehnavi-Arani, S.; Evazabadian, F.; Mahmoodian, V. An Ant Colony Algorithm (ACA) for solving the new integrated model of job shop scheduling and conflict-free routing of AGVs. *Comput. Ind. Eng.* **2015**, *86*, 2–13. [\[CrossRef\]](#)
53. Mousavi, M.; Yap, H.; Musa, S.; Tahriri, F.; Md Dawal, S. Multi-objective AGV scheduling in an FMS using a hybrid of genetic algorithm and particle swarm optimization. *PLoS ONE* **2017**, *12*, e0169817. [\[CrossRef\]](#)
54. Baruwa, O.; Piera, M. A coloured Petri net-based hybrid heuristic search approach to simultaneous scheduling of machines and automated guided vehicles. *Int. J. Prod. Res.* **2016**, *54*, 4773–4792. [\[CrossRef\]](#)
55. Vivaldini, K.; Rocha, L.; Martarelli, N.; Becker, M.; Moreira, A. Integrated tasks assignment and routing for the estimation of the optimal number of AGVs. *Int. J. Adv. Manuf. Technol.* **2016**, *82*, 719–736. [\[CrossRef\]](#)
56. Mudrova, L.; Hawes, N. Task scheduling for mobile robots using interval algebra. In Proceedings of the 2015 IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015; pp. 383–388. [\[CrossRef\]](#)
57. Dang, Q.V.; Rudová, H.; Nguyen, C.T. Adaptive Large Neighborhood Search for Scheduling of Mobile Robots. In Proceedings of the 2019 Genetic and Evolutionary Computation Conference (GECCO'19), Prague Czech Republic, 13–17 July 2019; LopezIbanez, M., Ed.; ACM, Inc.: New York, NY, USA, 2019; pp. 224–232. [\[CrossRef\]](#)
58. Al-Momani, H.; Al-Aubidv, K.M. Fuzzy-Based Task Scheduling of Mobile Robots in Flexible Manufacturing Systems. In Proceedings of the 2020 17th International Multi-Conference on Systems, Signals & Devices (SSD), Sfax, Tunisia, 20–23 July 2020; pp. 565–571. [\[CrossRef\]](#)
59. Dang, Q.; Nguyen, C.; Rudová, H. Scheduling of mobile robots for transportation and manufacturing tasks. *J. Heuristics* **2019**, *25*, 175–213. [\[CrossRef\]](#)
60. Bakshi, S.; Feng, T.; Yan, Z.; Chen, D. A Regularized Quadratic Programming Approach to Real-Time Scheduling of Autonomous Mobile Robots in a Prioritized Task Space. In Proceedings of the 2019 American Control Conference (ACC), Philadelphia, PA, USA, 10–12 July 2019; pp. 1361–1366. [\[CrossRef\]](#)
61. Cheng, B.; Xie, T.; Wang, L.; Tan, Q.; Cao, X. Deep reinforcement learning driven cost minimization for batch order scheduling in robotic mobile fulfillment systems. *Expert Syst. Appl.* **2024**, *255*, 124589. [\[CrossRef\]](#)
62. Yao, Y.; Wang, Q.; Wang, C.; Li, X.; Gao, L.; Xia, K. Knowledge-based multi-objective evolutionary algorithm for energy-efficient flexible job shop scheduling with mobile robot transportation. *Adv. Eng. Inform.* **2024**, *62*, 102647. [\[CrossRef\]](#)
63. Atik, S.T.; Chavan, A.S.; Grosu, D.; Brocanelli, M. A Maintenance-Aware Approach for Sustainable Autonomous Mobile Robot Fleet Management. *IEEE Trans. Mob. Comput.* **2024**, *23*, 7394–7407. [\[CrossRef\]](#)
64. Samsuria, E.; Mahmud, M.; Abdul Wahab, N.; Romdlony, M.; Zainal Abidin, M.; Buyamin, S. Adaptive fuzzy-genetic algorithm operators for solving mobile robot scheduling problem in job-shop FMS environment. *Robot. Auton. Syst.* **2024**, *176*, 104683. [\[CrossRef\]](#)
65. Li, Z.; Barenji, A.; Jiang, J.; Zhong, R.; Xu, G. A mechanism for scheduling multi robot intelligent warehouse system face with dynamic demand. *J. Intell. Manuf.* **2020**, *31*, 469–480. [\[CrossRef\]](#)
66. Dou, J.; Chen, C.; Yang, P. Genetic Scheduling and Reinforcement Learning in Multirobot Systems for Intelligent Warehouses. *Math. Probl. Eng.* **2015**, *2015*, 597956. [\[CrossRef\]](#)
67. Yi, S.; Luo, J. Heuristic Scheduling for Robotic Job Shops Using Petri Nets and Artificial Potential Fields. *IEEE Trans. Autom. Sci. Eng.* **2025**, *22*, 7556–7568. [\[CrossRef\]](#)

68. Leet, C.; Oh, C.; Lora, M.; Koenig, S.; Nuzzo, P. Task Assignment, Scheduling, and Motion Planning for Automated Warehouses for Million Product Workloads. In Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023; pp. 7362–7369. [\[CrossRef\]](#)
69. Boysen, N.; Schwerdfeger, S.; Ulmer, M. Robotized sorting systems: Large-scale scheduling under real-time conditions with limited lookahead. *Eur. J. Oper. Res.* **2023**, *310*, 582–596. [\[CrossRef\]](#)
70. Cechinel, A.; De Pieri, E. Centralized multi-robot logistic system: An approach using the island model genetic algorithm as task scheduler. *Int. J. Adv. Robot. Syst.* **2024**, *21*. [\[CrossRef\]](#)
71. Wang, J.; Zhang, Y.; Liu, Y.; Wu, N. Multiagent and Bargaining-Game-Based Real-Time Scheduling for Internet of Things-Enabled Flexible Job Shop. *IEEE Internet Things J.* **2019**, *6*, 2518–2531. [\[CrossRef\]](#)
72. Zhao, Y.; Zhu, K.; Song, X.; Zhang, J. An AGV Task Scheduling Method Based on Multi-Agent Reinforcement Learning. In Proceedings of the 2023 IEEE 12th Data Driven Control and Learning Systems Conference (DDCLS), Xiangtan, China, 12–14 May 2023; pp. 1504–1509. [\[CrossRef\]](#)
73. Sun, C.; Wang, Y.; Zhang, F.; Li, D.; Tan, Y.; Zhang, J. Chaotic Sparrow Search Algorithm Based Scheduling for Flexible Job Shop with Automatic Guided Vehicle. In Proceedings of the 2023 4th International Conference on Information Science, Parallel and Distributed Systems (ISPDS), Guangzhou, China, 12–14 March 2023; pp. 580–584. [\[CrossRef\]](#)
74. Geng, S.; Guo, Y.; Huang, S.; Sitahong, A. Multi-agent Deep Reinforcement Learning Based Integrated Scheduling of Machines and AGVs in Discrete Manufacturing Workshop. In Proceedings of the 2024 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS), Shah Alam, Malaysia, 29 June 2024; pp. 59–64. [\[CrossRef\]](#)
75. Dong, X.; Wan, G.; Zeng, P. Flexible job shop machines and AGVs cooperative scheduling on the basis of DQN algorithm. In Proceedings of the 2024 IEEE 6th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), Dalian, China, 16–18 August 2024; Volume 6, pp. 1808–1814. [\[CrossRef\]](#)
76. Abidi, M.H.; Alkhalefah, H.; Mohammed, M.K.; Umer, U.; Qudeiri, J.E.A. Optimal Scheduling of Flexible Manufacturing System Using Improved Lion-Based Hybrid Machine Learning Approach. *IEEE Access* **2020**, *8*, 96088–96114. [\[CrossRef\]](#)
77. Krishnamoorthy, P.; Satheesh, N.; Sudha, D.; Sengan, S.; Alharbi, M.; Pustokhin, D.A.; Pustokhina, I.V.; Setiawan, R. Effective Scheduling of Multi-Load Automated Guided Vehicle in Spinning Mill: A Case Study. *IEEE Access* **2023**, *11*, 9389–9402. [\[CrossRef\]](#)
78. Song, X.; Zhu, K.; Zhao, Y.; Zhang, J. Heterogeneous AGVs Scheduling in Hospital Using ALNS-based Metaheuristic Algorithm. In Proceedings of the 2023 IEEE 12th Data Driven Control and Learning Systems Conference (DDCLS), Xiangtan, China, 12–14 May 2023; pp. 495–500. [\[CrossRef\]](#)
79. Cui, Y.; Jia, B.; Sang, H.; Meng, L.; Zhang, B.; Zou, W. An Effective Discrete Jaya Algorithm for Multi-AGVs Scheduling Problem With Dynamic Unloading Time. *IEEE Access* **2024**, *12*, 101701–101716. [\[CrossRef\]](#)
80. Pan, Z.; Wang, L.; Zheng, J.; Chen, J.-F.; Wang, X. A Learning-Based Multipopulation Evolutionary Optimization for Flexible Job Shop Scheduling Problem with Finite Transportation Resources. *IEEE Trans. Evol. Comput.* **2023**, *27*, 1590–1603. [\[CrossRef\]](#)
81. Wang, M.; Xin, B. A Genetic Algorithm for Solving Flexible Flow Shop Scheduling Problem with Autonomous Guided Vehicles. In Proceedings of the 2019 IEEE 15th International Conference on Control and Automation (ICCA), Edinburgh, Scotland, UK, 16–19 July 2019; pp. 922–927. [\[CrossRef\]](#)
82. Feo-Flushing, E.; Gambardella, L.M.; Caro, G.A.D. Spatially-Distributed Missions With Heterogeneous Multi-Robot Teams. *IEEE Access* **2021**, *9*, 67327–67348. [\[CrossRef\]](#)
83. Yu, N.; Li, T.; Wang, B. Multi-load AGVs scheduling algorithm in automated sorting warehouse. In Proceedings of the 2021 14th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 11–12 December 2021; pp. 126–129. [\[CrossRef\]](#)
84. Lin, Z.; Ding, P.; Li, J. Task Scheduling and Path Planning of Multiple AGVs via Cloud and Edge Computing. In Proceedings of the 2021 IEEE International Conference on Networking, Sensing and Control (ICNSC), Xiamen, China, 3–5 December 2021; Volume 1, pp. 1–6. [\[CrossRef\]](#)
85. Ho, T.M.; Nguyen, K.-K.; Cheriet, M. Federated Deep Reinforcement Learning for Task Scheduling in Heterogeneous Autonomous Robotic System. *IEEE Trans. Autom. Sci. Eng.* **2024**, *21*, 528–540. [\[CrossRef\]](#)
86. Li, M.P.; Sankaran, P.; Kuhl, M.E.; Ptucha, R.; Ganguly, A.; Kwasinski, A. Task selection by autonomous mobile robots in a warehouse using deep reinforcement learning. In Proceedings of the WSC '19, National Harbor, MD, USA, 8–11 December 2020; pp. 680–688.
87. Dang, Q.V.; Martagan, T.; Adan, I.; Kleinlugtenbeld, J. Order Release Strategies for a Collaborative Order Picking System. In Proceedings of the WSC '22, Singapore, 11–14 December 2023; pp. 1521–1532.
88. Holland, J.H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*; University of Michigan Press: Ann Arbor, MI, USA, 1975.
89. Ropke, S.; Pisinger, D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transp. Sci.* **2006**, *40*, 455–472. [\[CrossRef\]](#)

90. Kennedy, J.; Eberhart, R. Particle swarm optimization. In Proceedings of the ICNN'95—International Conference on Neural Networks, Perth, WA, Australia, 27 November–1 December 1995; Volume 4, pp. 1942–1948. [[CrossRef](#)]
91. Bilge, Ü.; Ulusoy, G. A time window approach to simultaneous scheduling of machines and material handling system in an FMS. *Oper. Res.* **1995**, *43*, 1058–1070. [[CrossRef](#)]
92. Lawrence, S. *Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques*; Technical Report; Graduate School of Industrial Administration, Carnegie-Mellon University: Pittsburgh, PA, USA, 1984.
93. Brandimarte, P. Routing and scheduling in a flexible job shop by tabu search. *Ann. Oper. Res.* **1993**, *41*, 157–183. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.